

Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

September 6, 2026

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Problem Statement	5
1.3	Contributions	5
1.4	Thesis Outline	6
2	Regular Expressions	7
2.1	Syntax	7
2.2	Semantics	7
2.3	Rust Implementation	8
3	Derivatives and Partial Derivatives	10
3.1	Deciding Membership: An Overview	10
3.2	Brzozowski Derivatives	11
3.3	Antimirov Partial Derivatives	12
3.4	Matcher Selection	14
4	Matching versus Parsing	16
4.1	The Matching Problem	16
4.2	Parse Trees as Matching Evidence	16
4.3	Ambiguity in Regular Expression Parsing	17
4.3.1	Greedy Leftmost Matching	18
4.3.2	The Infinite Empty-Match Chain Problem	18
4.3.3	POSIX Longest-Leftmost Matching	19
5	Derivative-Based Parser	21
5.1	Core Algorithm	21
5.2	Correctness Properties	23
5.3	Bit-Coded Optimization (Theory)	23
5.4	Simplification Rules	26
5.5	Rust Implementation: The <code>deriv_std</code> Parser	26
5.5.1	<code>ParseTree</code> and <code>flatten</code>	27
5.5.2	<code>mk_eps</code>	27
5.5.3	<code>inject</code>	27
5.5.4	<code>parse_recursive</code> vs. <code>parse_loop</code>	27
5.6	Rust Implementation: The Bit-Coded Version	28
5.6.1	<code>ARegex</code> – Bit-Annotated Regex	28
5.6.2	<code>internalize/fuse</code>	28

5.6.3	<code>deriv_bc</code>	28
5.6.4	<code>mk_eps_bc</code>	29
5.6.5	<code>decode</code>	29
5.7	Worked Example: <code>paper_r1</code> End to End	29
6	Partial-Derivative-Based Parser	36
6.1	Extending Derivatives to Partial Derivatives	36
6.2	Bit-Coded Variant	37
6.3	Rust Implementation: The <code>pderiv_bc</code> Parser	38
6.4	Rust Implementation: The <code>pderiv_std</code> Parser	39
6.5	Worked Example: <code>paper_r1</code> End to End	41
7	Frontend: Parsing External Regular Expression Syntax	48
7.1	Two ASTs, One Lowering Step	48
7.2	Parsing Surface Syntax	49
7.3	Translating <code>ExtPat</code> into <code>Regex</code>	50
7.4	Faking Substring Search	51
8	Evaluation	53
8.1	Compared Engines	53
8.1.1	Rust’s <code>regex</code> Crate	53
8.1.2	<code>RE2</code>	54
8.2	Testing Strategy	54
8.3	Dataset Preparation	55
8.4	Correctness Verification	56
8.4.1	Paper Examples Reproduced	56
8.4.2	Cross-Implementation Equivalence	57
8.5	Performance Analysis	57
8.5.1	Stack Depth: Recursive vs. Iterative Construction	57
8.5.2	Heap Allocation: <code>deriv_std</code> vs. <code>deriv_bc</code>	58
8.5.3	Derivative-Based vs. Partial-Derivative-Based	58
8.5.4	Pathological Inputs	59
8.5.5	Benchmark Results	60
8.6	Comparison with Existing Engines	62
8.6.1	Performance	62
8.6.2	Standardized Benchmarking via <code>rebar</code>	64
9	Discussion	65
9.1	Discussion of Evaluation Results	65
9.2	Rust versus Haskell	66
9.2.1	Language Characteristics of the Reference Implementation	66
9.2.2	What the Reference Provides vs. What It Leaves Open	68
9.2.3	Translation Strategy	69
9.2.4	Translation Insights	69
9.3	Mismatch Analysis via Derivative Tracing	70
9.3.1	The Gap	70
9.3.2	Deriving Diagnostics for Free from the Algorithm Itself	70
9.3.3	Design	71
9.3.4	Scope and Limits	71

9.4	Optimization Challenges and Insights	71
9.5	Future Work	73
9.5.1	Paper-Suggested Extensions Not Yet Implemented	73
9.5.2	Further Performance Optimizations	74
9.5.3	A Full Mismatch Diagnostics System	76
10	Related Work and Conclusion	77
A	Code Listings	80
B	Full Benchmark Results	81
C	Test Case Catalogue	82
D	CLI and ENV Reference	83

Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the fundamental problem is regular expression matching: given a regular expression and an input string, determine whether the string belongs to the language denoted by the expression. Formally, this is the *membership problem*.

For example, the expression $(ab|a)^*$ matches the input string "aba", as it can be decomposed into "ab" followed by "a", but does not match "abb", since no sequence of subexpressions can produce that input.

For many applications, a Boolean answer is enough. An input validator may only need to accept or reject a password, URL, or configuration value. Other applications require more: a lexer must determine how an input decomposes into tokens, while capturing groups, schema validation, and information extraction require identifying which parts of the input correspond to different parts of the expression. In these settings, recognition alone is insufficient: a matcher must also recover the *structure* of a match, turning a membership problem into a parsing problem.

Efficiently solving these problems remains an active research area: the choice of algorithm influences performance, memory consumption, and security, and, once matching becomes parsing, the choice of how a match's structure is recovered influences correctness as well.

Existing approaches include backtracking, finite-automata-based techniques such as non-deterministic (NFA) and deterministic (DFA) automata. Each approach provides different trade-offs between implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely adopted, but may exhibit exponential running time on carefully constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require costly preprocessing and can suffer from state-space explosion.

Derivative-based algorithms provide an alternative by transforming the regular expression itself as input symbols are consumed. Each derivative represents the language that remains to be matched after a particular input symbol has been consumed. By maintaining a direct correspondence between an expression and its successive residuals, derivative-based methods provide an elegant foundation for regular expression matching and, with suitable extensions, structured parsing.

However, derivative-based techniques are less commonly used in practice than other matching approaches. Bridging this gap "From Theory to Tools" motivates the work presented in this thesis.

1.2 Problem Statement

Brzozowski introduced regular expression derivatives as an algebraic characterization of regular expression matching [Brzozowski, 1964]. Antimirov later introduced partial derivatives, representing sets of residual expressions and provide a construction closely related to nondeterministic automata while reducing redundancy compared with ordinary derivatives [Antimirov, 1996]. Building on these ideas, Sulzmann and Lu presented a derivative-based algorithm for POSIX-correct regular expression parsing [Sulzmann and Lu, 2014].

Although the algorithm is formally developed and accompanied by a Haskell reference implementation, its practical behaviour can be further studied through an implementation in a systems programming language. Rust provides such an environment through its compile-time ownership model, allowing memory allocation, stack usage, and execution time to be studied as direct properties of the implementation.

This thesis investigates the implementation of Sulzmann and Lu's POSIX derivative-based parsing algorithm in Rust and explores an extension of this construction based on Antimirov's partial derivatives. The implementations also incorporate the bit-coded optimization proposed by the same authors, to reduce memory usage and improve performance.

The resulting implementations are evaluated in terms of performance and resource usage, both against each other and on real-world regular expression patterns drawn from network-signature, spam-filtering, and general-purpose corpora (Chapter 8). The evaluation also includes a direct empirical comparison with two established engines: Rust's `regex` crate and Google's RE2.

Furthermore, the derivative-based representation provides an opportunity to show the reasoning of the algorithm. This thesis therefore also shows how derivative computations can be used as a basis for diagnostic tooling, allowing matches and mismatches to be explained through the transformations performed during the matching process.

1.3 Contributions

This thesis makes the following contributions:

- **A Rust implementation of Sulzmann and Lu's derivative-based POSIX parser**, including a direct two-pass construction following the original algorithm and a bit-coded single-pass optimization. The implementation additionally explores partial derivatives as an alternative representation, resulting in four parser variants: derivative-based and partial-derivative-based, in direct and bit-coded forms.
- **An empirical evaluation** comparing all four parser combinations with each other, on both synthetic benchmarks and a triaged corpus of real-world regular expressions, based on the datasets used in the paper of Mamouras et al. [Mamouras et al., 2024].

Then comparing them against established regular expression matchers such as Rust’s `regex` crate and Google’s RE2.

- **A derivative-tracing diagnostics system**, showing the parser’s computation steps to explain successful matches and mismatches, to provide a tool for understanding the algorithm’s behaviour and debugging of regular expressions.

1.4 Thesis Outline

The remainder of this thesis is structured as follows. Section 2 introduces the syntax and semantics of regular expressions, together with the Rust representation of their abstract syntax used throughout the rest of this work. Section 3 builds on this representation to introduce Brzozowski’s derivatives and Antimirov’s partial derivatives as algorithms for deciding language membership, along with their Rust implementation – the foundation on which the parsers presented later are built.

Before extending these algorithms to parsing, Section 4 distinguishes matching from parsing and introduces the disambiguation problem that arises once a parse tree, not just a boolean, is required. Section 5 then presents the derivative-based POSIX parser adopted as this thesis’s algorithmic basis, Sulzmann and Lu’s construction [Sulzmann and Lu, 2014], together with its Rust implementation in both a direct ("inj") and a bit-coded form. Section 6 extends this construction from Brzozowski derivatives to Antimirov’s partial derivatives, again in both its direct and bit-coded variants.

With all four parser variants in place, Section 8 evaluates their correctness and performance against each other, on both synthetic benchmarks and a corpus of real-world regular expressions, and positions this evaluation relative to established regular expression engines. Section 9 situates this work relative to related approaches, reflects on translating the underlying algorithm from Haskell to Rust, and discusses the derivative-tracing facility built alongside the parser, their usefulness and potential extensions.

Section 10 closes by summarizing the main findings and contributions of the thesis and mentioning of related work.

Regular Expressions

2.1 Syntax

A regular expression is built from a small set of primitive constructs, combined by three operators. Over an alphabet Σ of symbols, the syntax is given by the following grammar:

$$r, s ::= x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \quad (x \in \Sigma)$$

Here x denotes a literal symbol, ε the expression matching only the empty word, and $\emptyset$ the expression matching no word at all. The operators $+$, $\cdot$, and $*$ denote alternation, concatenation, and repetition (Kleene star) respectively; concatenation is frequently left implicit, writing rs for $r \cdot s$.

Unless parentheses are shown explicitly, Kleene star binds most tightly, followed by concatenation, followed by alternation. Thus, for example, $ab^* + c$ is parsed as $(a(b^*)) + c$.

As a running example, consider the problem of recognizing binary strings that end in "01" – a small but non-trivial pattern that touches every construct in the grammar above. Over $\Sigma = \{0, 1\}$, this pattern is expressed as

$$r_{01} = (0 + 1)^* \cdot 0 \cdot 1.$$

Reading this expression against the grammar: $(0 + 1)$ is an alternation of two literals, $(0 + 1)^*$ applies Kleene star to that alternation, and the result is concatenated with the literals 0 and 1 in sequence.

2.2 Semantics

The syntax above describes the *shape* of an expression; the semantics assigns each expression the *language* it denotes. For a regular expression r , write $L(r)$ for the set of words it matches. The semantics is defined by structural recursion on r :

$$\begin{aligned}
L(x) &= \{x\} \\
L(\varepsilon) &= \{\varepsilon\} \\
L(\emptyset) &= \{\} \\
L(r + s) &= L(r) \cup L(s) \\
L(r \cdot s) &= \{vw \mid v \in L(r), w \in L(s)\} \\
L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid w_1, \dots, w_n \in L(r), n \geq 1\}
\end{aligned}$$

For example, these following examples illustrate the distinction between an expression and the language it denotes: different expressions may describe related or even identical sets of words, while the semantics assigns a precise language to each expression.

Expression	Language
a	$\{a\}$
ε	$\{\varepsilon\}$
$\emptyset$	$\emptyset$
$a + b$	$\{a, b\}$
ab	$\{ab\}$
a^*	$\{\varepsilon, a, aa, aaa, \dots\}$

A word w *matches* r exactly when $w \in L(r)$ – this is the membership problem introduced in Section 1.1.

Unwinding the semantics for the running example, $L(0 + 1) = \{0, 1\}$, so $L((0 + 1)^*)$ is the set of all finite binary strings, including the empty string. Concatenating with $L(0) = \{0\}$ and then $L(1) = \{1\}$ restricts this to

$$L(r_{01}) = \{w \cdot 0 \cdot 1 \mid w \in \{0, 1\}^*\},$$

i.e. exactly the binary strings ending in "01" – "01", "001", "1101", and so on. In particular, the empty string is not matched, since every word in $L(r_{01})$ contains the final literal 0 followed by the literal 1.

2.3 Rust Implementation

The grammar above is represented directly as a Rust enum, mirroring each syntactic construct one to one:

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi,
    Eps,
    Lit(char),
    Seq(Box<Regex>, Box<Regex>),
    Alt(Box<Regex>, Box<Regex>),
    Star(Box<Regex>),
}
```

For example, the expression

$$(0 + 1)^*01$$

is represented structurally by an expression tree whose root is a concatenation, with $(0+1)^*$ and 01 as its subexpressions. More explicitly, its structure can be viewed schematically as

$$\text{Seq}(\text{Seq}(\text{Star}(\text{Alt}(0, 1)), 0), 1).$$

Thus, the Rust value is not merely a string representation of the regular expression; it is a tree that directly represents its syntactic structure. This structural representation is what later allows operations to be defined recursively over the expression.

This one-to-one correspondence is not incidental. Rust’s enums are algebraic data types: each variant corresponds to exactly one production of the grammar, and Rust’s `match` requires every variant to be handled, rejecting the program at compile time otherwise. This gives the same guarantee an inductive definition of r in Section 2.1 carries on paper – that a function defined by cases on r is defined for every possible r – as a property the compiler checks, rather than one that must be checked by inspection. This is a structural fit shared with other languages that support algebraic data types, including Haskell itself; Rust’s specific contribution is not this correspondence but what it costs to maintain, which is not always nothing. `Phi` and `Eps` carry no data, matching $L(\emptyset) = \{\}$ and $L(\varepsilon) = \{\varepsilon\}$ directly, and `Lit` carries the literal symbol x – these three variants transcribe directly with no adjustment.

The recursive constructs – `Seq`, `Alt`, and `Star` – require explicit indirection in Rust. A type must have a known size at compile time, so a recursive definition such as `Seq(Regex, Regex)` would have infinite size: each `Regex` would contain another `Regex`, and so on. `Box<Regex>` breaks this recursion by storing the subexpression separately and keeping only a fixed-size pointer in the enum. The need for this explicit indirection is one of the practical differences between the Rust implementation and the Haskell reference implementation, and is revisited more generally in Section 9.2.

Derivatives and Partial Derivatives

Chapter 2 fixed the syntax and semantics of regular expressions: $L(r)$ tells us, declaratively, which words r matches. It says nothing about how to compute membership. This chapter introduces three ways of doing so – naive recursive backtracking, and two algebraic reformulations due to Brzozowski and Antimirov – each realized as a Rust matcher with the common signature `fn(&str, &Regex) -> bool`. All three answer exactly the membership question of Section 2.2; they differ in how directly that question is asked of the input, and in what it costs to ask it repeatedly. The derivative operation introduced here is also the foundation the POSIX parsers of Chapters 5 and 6 are built on – membership is a special case of parsing that discards the parse tree, and it is the simplest setting in which to introduce the operation itself.

3.1 Deciding Membership: An Overview

The most direct way to decide $w \in L(r)$ is to read $L(r)$'s definition as an algorithm: try every way w could have been produced by r 's structure, and succeed if any of them works. Writing w as an indexable sequence of characters and (i, j) for the substring between positions i and j , membership of the whole word corresponds to $(0, |w|)$, and each case of $L(r)$ becomes a recursive case of a function `matchR` deciding whether r matches the substring (i, j) :

- ε matches (i, j) iff $i = j$.
- A literal x matches (i, j) iff $j = i + 1$ and the character at position i is x .
- $r + s$ matches (i, j) iff r matches (i, j) or s does.
- $r \cdot s$ matches (i, j) iff there is some split point $k \in [i, j]$ such that r matches (i, k) and s matches (k, j) .
- r^* matches (i, j) iff $i = j$, or there is some $k \in (i, j]$ such that r matches (i, k) and r^* matches (k, j) .

This is a correct matcher by construction: every case reproduces the corresponding case of $L(r)$ verbatim, so there is nothing to prove beyond the definition itself. It is also, in general, an expensive one, because concatenation and star both branch over *every* split point in the current range rather than committing to one, and nothing remembers a sub-range once it has been explored from a different call.

To see the blowup concretely, take $r = a \cdot a \cdot a$ (three literals concatenated, right-associated as $a \cdot (a \cdot a)$) against $w = \mathbf{aaa}$. Matching $(0, 3)$ against r tries every split point $k \in \{0, 1, 2, 3\}$

for the outer concatenation, but only $k = 1$ can possibly succeed, since the left operand is a single literal and can only match a range of length exactly 1; the other three attempts are wasted work discovered only after descending into them. The right operand at that point, $(1, 3)$ against $a \cdot a$, repeats the same pattern one level down, again trying every split point in $\{1, 2, 3\}$ before finding $k = 2$. Each concatenation node re-explores every split point of the range handed to it with no memory of the fact that the same range, or a sub-range of it, may already have been tried while resolving a sibling call – for a chain of n literals against an input of length n , the number of (i, j) pairs examined this way grows exponentially in n rather than linearly. `src/matchers/match_naive.rs` transcribes this scheme directly, operating on a `Vec<char>` and a pair of `usize` bounds in place of the Haskell reference’s `matchR w (i, j) r`; the benchmarks in Chapter 8 restrict this matcher to inputs of length 15 or less precisely because of this blowup.

`match_naive` is nonetheless useful, and stays in the codebase for that reason: because it is a direct transcription of $L(r)$ ’s definition, it serves as an independent oracle that the more efficient matchers introduced below can be checked against – any disagreement is a bug in one of the newer matchers, not a question of what the "correct" answer is supposed to be. The remainder of this chapter develops two such matchers, both based on a different idea entirely – instead of searching for a decomposition of w against a fixed r , *transform* r itself as each character of w is consumed, so that membership of the remaining suffix is asked against a smaller, adjusted expression rather than re-asked against the same r every time.

3.2 Brzowski Derivatives

Brzowski’s derivative of r with respect to a symbol x , written $r \backslash x$, is the expression describing exactly what remains of r ’s language after the leading symbol x has been removed:

$$L(r \backslash x) = x \backslash L(r) = \{ w \mid xw \in L(r) \} \text{ [Brzowski, 1964].}$$

Deciding whether a word $x_1 x_2 \cdots x_n$ is in $L(r)$ then reduces to computing the chain $r \backslash x_1 \backslash x_2 \backslash \cdots \backslash x_n$ and checking whether the final expression accepts the empty word – i.e. whether it is *nullable*. Both nullability and the derivative itself are defined by structural recursion directly on the syntax of r , with no reference to $L(r)$ needed once the base cases are fixed:

$$\begin{array}{lll} n(\emptyset) = \text{false} & n(\varepsilon) = \text{true} & n(x) = \text{false} \\ n(r + s) = n(r) \vee n(s) & n(r \cdot s) = n(r) \wedge n(s) & n(r^*) = \text{true} \end{array}$$

$$\begin{array}{lll} \emptyset \backslash x = \emptyset & \varepsilon \backslash x = \emptyset & y \backslash x = \begin{cases} \varepsilon & x = y \\ \emptyset & x \neq y \end{cases} \\ (r + s) \backslash x = r \backslash x + s \backslash x & r^* \backslash x = (r \backslash x) \cdot r^* & \\ (r \cdot s) \backslash x = \begin{cases} (r \backslash x) \cdot s + s \backslash x & n(r) \\ (r \backslash x) \cdot s & \text{otherwise} \end{cases} & & \end{array}$$

The only case that needs justification beyond direct case analysis on $L(r \cdot s)$ is concatenation: if r is nullable, a word starting with x can either continue matching r (so the

derivative goes into r while s is left untouched) or arise from r already being satisfied by ε and x belonging to s instead – hence the extra $s \setminus x$ summand exactly when $n(r)$ holds. If r is not nullable, the second possibility cannot arise, since $\varepsilon \notin L(r)$ rules out r having already been satisfied before any input was consumed.

Worked example: single-symbol matching. Consider x^* against the word $\mathbf{xx}$. Writing $r \xrightarrow{x} r \setminus x$ for one derivative step,

$$x^* \xrightarrow{x} (x \setminus x) \cdot x^* = \varepsilon \cdot x^* \xrightarrow{x} (\varepsilon \setminus x) \cdot x^* + x^* \setminus x = \emptyset \cdot x^* + (x \setminus x) \cdot x^* = \emptyset \cdot x^* + \varepsilon \cdot x^*.$$

Checking nullability of the final expression by structural recursion,

$$\begin{aligned} n(\emptyset \cdot x^* + \varepsilon \cdot x^*) &= n(\emptyset \cdot x^*) \vee n(\varepsilon \cdot x^*) \\ &= (n(\emptyset) \wedge n(x^*)) \vee (n(\varepsilon) \wedge n(x^*)) \\ &= (\text{false} \wedge \text{true}) \vee (\text{true} \wedge \text{true}) = \text{true}, \end{aligned}$$

so x^* matches $\mathbf{xx}$. Two things are worth noting in this short trace. First, the expression has grown from one node to five across two steps, purely from the $+$ introduced at each nullable-prefix step of the concatenation rule; nothing above bounds this growth, and a third derivative step would nest a further $+$ inside each existing summand. Second, applying the algebraic identities $\emptyset \cdot r = \emptyset$ and $\emptyset + r = r$ collapses the final expression back down to $\varepsilon \cdot x^*$ – exactly the expression reached after only *one* derivative step. This is not a coincidence particular to this example: it is the general phenomenon Section 5.4 returns to for the bit-coded construction, where the same a^* -style blowup is shown to collapse back to a single equivalence class at every step once simplification is applied.

`src/regex/nullable.rs` and `src/regex/deriv.rs` transcribe $n(r)$ and $r \setminus x$ case for case, matching the `Regex` enum of Section 2.3 one variant at a time. `match_deriv` (`src/matchers/match_deriv.rs`) folds `deriv` over the input characters and checks `nullable` on the result, exactly as `matchDeriv` does in the reference implementation. One deliberate departure from that reference: after every derivative step, `match_deriv` also applies `simplify` (`src/regex/simplify.rs`, discussed further in Section 5.4), collapsing exactly the algebraic identities used informally above – $\emptyset \cdot r = \emptyset$, $\varepsilon \cdot r = r$, $\emptyset + r = r$, and idempotent alternation $r + r = r$. Applied to the trace above, `simplify` would keep the expression at $\varepsilon \cdot x^*$ (equivalently, after normalizing $\varepsilon \cdot r$ to r , at x^* itself) after every step rather than letting it grow. Since `simplify` preserves $L(r)$ by construction, this changes nothing about which words are accepted; it only keeps the expressions produced along the way from growing as in the example above.

The sequence of derivatives $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \dots \xrightarrow{x_n} r_n$ can be read as tracing a path through an automaton built lazily, on demand: each r_i is a state, and $\xrightarrow{x}$ is the transition relation. This "derivatives as an implicit DFA" reading is what motivates calling `match_deriv` DFA-style throughout this thesis – at every step there is exactly one current expression, playing the role of exactly one current automaton state, however large that single state's internal syntax tree happens to be.

3.3 Antimirov Partial Derivatives

The derivative of Section 3.2 keeps the residual language as a single expression, at the cost of that expression's internal structure growing with every step – the $+$ introduced by

the concatenation rule nests deeper each time a nullable prefix is derived again, and only an explicit simplification pass folds it back down. Antimirov's partial derivatives take a different route to the same residual language: rather than combining the alternatives produced along the way with $+$, keep them apart as a *set* of expressions, one per strand of nondeterminism [Antimirov, 1996]. Writing $pd(r, x)$ for this set,

$$\begin{aligned}
pd(\emptyset, x) &= \{\} \\
pd(\varepsilon, x) &= \{\} \\
pd(y, x) &= \begin{cases} \{\varepsilon\} & x = y \\ \{\} & x \neq y \end{cases} \\
pd(r + s, x) &= pd(r, x) \cup pd(s, x) \\
pd(r^*, x) &= \{ smartC(r', r^*) \mid r' \in pd(r, x) \} \\
pd(r \cdot s, x) &= \begin{cases} \{ smartC(r', s) \mid r' \in pd(r, x) \} \cup pd(s, x) & n(r) \\ \{ smartC(r', s) \mid r' \in pd(r, x) \} & \text{otherwise} \end{cases}
\end{aligned}$$

where the smart constructor $smartC(\varepsilon, s) = s$ and $smartC(r', s) = r' \cdot s$ otherwise normalizes away the spurious $\varepsilon \cdot s$ that would otherwise appear whenever a partial derivative of r is exactly ε . This normalization is not just cosmetic: it is what guarantees every partial derivative is literally a subexpression of r (up to the concatenations $smartC$ builds), rather than an expression that could in principle grow without bound the way plain $r \setminus x$ can.

Worked example: derivative versus partial derivative, side by side. Take $r = a + ab$ and derive with respect to **a** under both constructions. Brzowski's derivative, following Section 3.2, produces one expression:

$$r \setminus \mathbf{a} = (a \setminus \mathbf{a}) + (ab \setminus \mathbf{a}) = \varepsilon + ((a \setminus \mathbf{a}) \cdot b) = \varepsilon + (\varepsilon \cdot b),$$

which **simplify** would reduce to $\varepsilon + b$ – still a *single* expression, whose top-level shape is an alternation joining the two strands together. Antimirov's partial derivative keeps those same two strands apart instead of joining them with $+$:

$$\begin{aligned}
pd(r, \mathbf{a}) &= pd(a, \mathbf{a}) \cup pd(ab, \mathbf{a}) \\
&= \{\varepsilon\} \cup \{ smartC(r', b) \mid r' \in pd(a, \mathbf{a}) \} \\
&= \{\varepsilon\} \cup \{ smartC(\varepsilon, b) \} = \{\varepsilon, b\}.
\end{aligned}$$

Both describe the same residual language, $\{\varepsilon\} \cup \{\mathbf{b}w \mid w \in \Sigma^*\}$ restricted to $\{\mathbf{b}\}$ itself here – but $r \setminus \mathbf{a}$ represents it as one expression with an internal $+$, while $pd(r, \mathbf{a})$ represents it as two separate, syntactically smaller expressions with no $+$ at all. This is the concrete sense in which pd is NFA-shaped: $\{\varepsilon, b\}$ is exactly the state set an NFA for r would be in after reading **a**, with one state per strand, rather than one DFA state encoding both strands internally.

That subexpression property underlies Antimirov's central finiteness result: the set of *all* partial derivatives reachable from r by derivation with respect to arbitrary words is finite, bounded by the number of literal-symbol occurrences in r [Antimirov, 1996]. For $r = a + ab$ above, there are three literal occurrences (the two a 's and the one b), so at most three *distinct* partial derivatives (plus r itself) can ever appear, no matter

how long a word r is derived with respect to $-$ and indeed $pd(r, a) = \{\varepsilon, b\}$ already uses two of that budget. Ordinary derivatives are also finite up to language equivalence, but only after applying simplification laws such as those of Section 5.4 to identify equivalent expressions; the partial-derivative bound holds already at the syntactic level, with no equivalence-checking required.

pd is implemented directly inside `match_pderiv` (`src/matchers/match_pderiv.rs`) as a private function returning `HashSet<Regex>`, reusing the same `smart_seq` smart constructor already introduced for `simplify` in Section 3.2 – `smartC` and `smart_seq` coincide, since both exist to normalize $\varepsilon \cdot r$ to r . Unlike `deriv` and `simplify`, this plain, un-annotated pd has no independent module of its own: no parser in Chapter 6 calls it directly – each parser variant there needs partial derivatives paired with parse-tree or bit information pd itself does not carry, and defines its own richer version of the same recursion instead (Section 6.3) – so `match_pderiv` is the only caller, and the function lives where it is used.

The matcher itself, `match_pderiv` in `src/matchers/match_pderiv.rs`, mirrors `matchPDeriv` from the reference implementation: rather than tracking one expression, it maintains a `HashSet<Regex>` of currently-live residuals, replacing that set at each character with the union of $pd(r', x)$ over every residual r' currently in it, and accepting once the input is exhausted if any surviving residual is nullable. Running `match_pderiv` on $r = a + ab$ against `ab` starts from $\{r\}$, steps to $pd(r, a) = \{\varepsilon, b\}$ exactly as computed above, then steps again on `b`: $pd(\varepsilon, b) \cup pd(b, b) = \{\} \cup \{\varepsilon\} = \{\varepsilon\}$, which contains a nullable residual, so the word is accepted. This is exactly subset simulation of an NFA whose states are subexpressions of the original r , run directly on the syntax rather than on a separately constructed automaton.

3.4 Matcher Selection

`match_naive`, `match_deriv`, and `match_pderiv` share the signature `fn(&str, &Regex) -> bool` despite implementing three structurally different algorithms – exhaustive search over substring splits, a single evolving DFA-style expression, and an evolving NFA-style set of expressions. This uniform interface is what makes the three matchers directly interchangeable: the same regular expression and input can be run through any of them, and any disagreement between their answers points to a bug in one of the three independently-written implementations rather than to an ambiguity in what the "correct" answer is supposed to be. This is precisely the property the matcher-agreement tests in the test suite exploit, and the same pattern is reused throughout this thesis for parsers rather than matchers, wherever multiple independently-derived implementations are expected to agree.

`MatcherType` (`src/matchers/selection.rs`) enumerates the three matchers and gives the `regex-engine` command-line tool a single place to resolve "which matcher runs" for a given invocation, via an explicit `--matcher` flag on the `match` subcommand (`naive`, `deriv`, `pderiv`, or `all`, defaulting to `deriv`) parsed through `clap`'s own enum derivation rather than a hand-written string match – there is exactly one selection mechanism, not two that could silently disagree. The standalone demo under `examples/` (`demo_match.rs`) takes no such flag or environment variable at all: it always runs every matcher unconditionally on a fixed set of worked examples, which is the simpler, and in this case sufficient, choice for a demo whose entire purpose is showing the three side by side. All three matchers can

also be compared side by side on an arbitrary input through the CLI itself, in a single invocation:

```
cargo run -- match "a*" "aaa" --matcher all
```

prints the result of `match_naive`, `match_deriv`, and `match_pderiv` together, letting the three be compared directly rather than one at a time.

The POSIX parsers built in Chapters 5 and 6 follow the same pattern one level up: rather than a boolean, each of the parser combinations introduced there returns a parse tree, but all of them are again selectable through a single uniform interface and are expected to agree with one another wherever they overlap.

Matching versus Parsing

Chapter 3 answered the membership question, always with a Boolean. This chapter motivates and makes precise the harder question the rest of this thesis is about: not just *whether* r matches w , but *how* – which parts of w were consumed by which parts of r . That question turns out not to have a unique answer in general, and this chapter’s second half develops the disambiguation machinery, in particular the POSIX policy, needed to make it one again.

4.1 The Matching Problem

The three matchers of Chapter 3 all decide the same question: given r and w , is $w \in L(r)$? A Boolean answer is enough for some uses of regular expressions – validating that a password meets a policy, or that a configuration value has an acceptable shape, needs nothing more. It is not enough for others. A lexer must know not just that an input decomposes into valid tokens, but exactly where each token begins and ends. Capturing groups, schema validation, and information extraction all require identifying which *part* of w corresponds to which *part* of r , not merely that some correspondence exists. In each of these settings the question is no longer recognition but parsing: recover a witness that $w \in L(r)$, structured enough to read off submatch boundaries from, rather than a single bit reporting that such a witness exists.

4.2 Parse Trees as Matching Evidence

Sulzmann and Lu formalize such a witness by viewing regular expressions as types and parse trees as the values inhabiting them [Sulzmann and Lu, 2014]. A parse tree v is built from the same constructors used informally in Section 2.2’s definition of $L(r)$ – a pair for concatenation, a tagged left or right injection for alternation, a list for repetition – and a judgment $\vdash v : r$, defined by structural recursion on both v and r simultaneously, states that v is a valid parse tree of r :

$$\begin{array}{c}
 \vdash () : \varepsilon \qquad \frac{l \in \Sigma}{\vdash l : l} \\
 \\
 \frac{\vdash v_1 : r_1 \quad \vdash v_2 : r_2}{\vdash (v_1, v_2) : r_1 r_2} \quad \frac{\vdash v_1 : r_1}{\vdash \text{Left } v_1 : r_1 + r_2} \quad \frac{\vdash v_2 : r_2}{\vdash \text{Right } v_2 : r_1 + r_2} \\
 \\
 \frac{}{\vdash [] : r^*} \quad \frac{\vdash v : r \quad \vdash vs : r^*}{\vdash (v : vs) : r^*}
 \end{array}$$

A *flattening* function $|v|$ reads the underlying word back off a parse tree by structural recursion – $|()| = \varepsilon$, $|l| = l$, $|(v_1, v_2)| = |v_1||v_2|$, $|Left\ v| = |Right\ v| = |v|$, and $|v : vs| = |v||vs|$ – and the two are related by the property that matters: for every derivable $\vdash v : r$, the word $|v|$ is a member of $L(r)$. A parse tree is thus not merely evidence that $w \in L(r)$; each derivation of $\vdash v : r$ with $|v| = w$ is a proof of that membership, in the same sense a value of a type is evidence that the type is inhabited.

Warm-up example. Before turning to a compound expression, it is worth seeing the rules apply to the simplest nontrivial case. For $r = a \cdot b$ against $w = \mathbf{ab}$, only one derivation is possible: the *Pair* rule requires $\vdash v_1 : a$ and $\vdash v_2 : b$, and the only parse trees of a single literal are the literal itself, so $v_1 = a$, $v_2 = b$, and

$$\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}$$

is the only possible derivation, giving $v = (a, b)$ with $|v| = |a||b| = \mathbf{ab}$. Nothing here is ambiguous yet – ambiguity, taken up in Section 4.3, requires a choice point such as $+$ or $*$ somewhere in r .

Compound example. Now take $r_1 = (a + ab)(b + \varepsilon)$ against the word $\mathbf{ab}$ (`paper_r1` in the test suite, `tests/common/mod.rs`, reproducing the running example of [Sulzmann and Lu, 2014]). One valid derivation picks the second alternative of $a + ab$, matching $\mathbf{ab}$ in one step, then the empty alternative of $b + \varepsilon$:

$$\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab} \Bigg/ \frac{\vdash (a, b) : a + ab}{\vdash Right(a, b) : a + ab} \quad \frac{}{\vdash () : \varepsilon} \Bigg/ \frac{\vdash () : \varepsilon}{\vdash Right() : b + \varepsilon}$$

yielding $v = (Right(a, b), Right())$ with $|v| = |(a, b)||()| = \mathbf{ab} \cdot \varepsilon = \mathbf{ab}$. A second, equally valid derivation exists for the same r_1 and the same word – Section 4.3 constructs it explicitly, since it is exactly what makes disambiguation necessary in the first place.

`src/types/tree.rs` represents v directly as the `ParseTree` enum – `Empty`, `Char`, `Pair`, `Left`, `Right`, `Star` correspond one to one with $()$, a literal, (v_1, v_2) , $Left\ v$, $Right\ v$, and $v:vs$ respectively (Section 5.5.1 presents the Rust type itself, alongside the rest of the `inj` parser it belongs to), and `flatten` is $|v|$ by structural recursion, matching each `ParseTree` case to the corresponding clause above. One constructor present in the reference's `U` type has no counterpart here: `Nil`, used there for the (untypeable) "no parse tree" case when a match does not exist. The Rust implementation has no need for it, representing absence of a parse tree with `Option<ParseTree>::None` at the type level instead – a first, small instance of a translation choice Rust's type system invites that the untyped reference does not; Section 9.2.3 returns to this pattern more generally once the full implementation is on the table.

4.3 Ambiguity in Regular Expression Parsing

The judgment $\vdash v : r$ is not functional: for a fixed r and w , there can be more than one v with $\vdash v : r$ and $|v| = w$. Return to $r_1 = (a + ab)(b + \varepsilon)$ against $\mathbf{ab}$ from Section 4.2: alongside $v = (Right(a, b), Right())$, the *first* alternative of $a + ab$ also leads to a valid,

different derivation – match **a** against the first alternative, leaving **b** for $b + \varepsilon$'s own first alternative:

$$\frac{}{\vdash a : a} \Big/ \frac{\vdash a : a}{\vdash \text{Left } a : a + ab} \quad \frac{}{\vdash b : b} \Big/ \frac{\vdash b : b}{\vdash \text{Left } b : b + \varepsilon}$$

giving $v' = (\text{Left } a, \text{Left } b)$, with $|v'| = |a||b| = \mathbf{ab}$ – the same word, a different parse. A second example makes the same point with repetition instead of concatenation: $r_2 = (a + b + ab)^*$ against **ab** (`paper_r2`¹ in the test suite). One iteration of the star can consume **ab** in a single step via the third alternative, giving $v = [\text{Right } \text{Right}(a, b)]$; alternatively, two iterations can consume **a** and **b** separately via the first two alternatives, giving $v' = [\text{Left } a, \text{Right } \text{Left } b]$. Nothing in Section 4.2's typing rules prefers one derivation over the other in either example. A parser that must return a single answer therefore needs an explicit *disambiguation policy*: a rule that, given r and w , picks out one v from the (possibly many) parse trees satisfying $\vdash v : r$ and $|v| = w$.

4.3.1 Greedy Leftmost Matching

The policy implicit in backtracking engines is *greedy*: alternatives are tried in left-to-right order and the first one that leads to an overall match wins, and inside a Kleene star, one more iteration is always attempted before the star is allowed to stop [Frisch and Cardelli, 2004]. Operationally this is exactly what a backtracking matcher's exploration order already realizes; formalized as an ordering on parse trees, the relevant rule states that between two candidates agreeing everywhere else, $\text{Left } v \geq_{r_1+r_2} \text{Right } v'$ always – the left alternative is preferred outright, independent of how long a match each side produces.

Applied to $r_1 = (a + ab)(b + \varepsilon)$ against **ab** from above, Greedy matching tries the first alternative of $a + ab$ first: a matches the leading character, so the engine commits to it immediately without ever considering whether ab might have matched more. What remains, **b**, is then matched against $b + \varepsilon$, whose own first alternative b succeeds directly. The Greedy result is therefore $v' = (\text{Left } a, \text{Left } b)$ – the *second* of the two derivations found in Section 4.3. Section 4.3.3 revisits this exact pair $(r_1, \mathbf{ab})$ once the POSIX ordering has been introduced, and finds the opposite answer.

4.3.2 The Infinite Empty-Match Chain Problem

Greedy's "always iterate before stopping" rule interacts badly with expressions that can match the empty word inside a star. Take ε^* against the empty input: since ε itself is nullable, nothing stops the star from iterating arbitrarily many times while still matching only ε overall, and Greedy's preference for "one more iteration" ranks every such expansion above the last:

$$v_0 = [] <_{\varepsilon^*} v_1 = [()] <_{\varepsilon^*} v_2 = [(), ()] <_{\varepsilon^*} \dots$$

This chain has no maximum, so Greedy matching on ε^* is only well-defined once such *problematic* expressions are given special treatment beyond the ordering rule of Section 4.3.1 alone. The same issue appears in slightly less degenerate form for $(\varepsilon + a)^*$ against **a**, where $v_0 = [\text{Right } a]$, $v_1 = [\text{Left } (), \text{Right } a]$, $v_2 = [\text{Left } (), \text{Left } (), \text{Right } a]$, ... again form an infinite ascending chain under Greedy, this time padding an arbitrary number of empty iterations in front of the one iteration that actually consumes **a**. Section 4.3.3

¹As implemented, `paper_r2` nests the alternation as $a + (b + ab)$; the associativity of $+$ is immaterial to $L(r)$ and to this example.

shows exactly how POSIX’s rule (K1) blocks both chains at v_0 directly, with no special case required.

4.3.3 POSIX Longest-Leftmost Matching

The POSIX standard specifies a different policy, *leftmost-longest* matching:

“Subpatterns should match the longest possible substrings, where subpatterns that start earlier (to the left) in the regular expression take priority over ones starting later. Hence, higher-level subpatterns take priority over their lower-level component subpatterns. Matching an empty string is considered longer than no match at all.” [IEEE and The Open Group, 2017]

Sulzmann and Lu formalize this as an ordering $\geq_r$ on parse trees of a fixed r , obtained from the Greedy order of Section 4.3.1 by replacing exactly one rule and leaving the rest unchanged [Sulzmann and Lu, 2014]:

$$\begin{aligned}
\text{(C1)} \quad & \frac{v_1 = v'_1 \quad v_2 \geq_{r_2} v'_2}{(v_1, v_2) \geq_{r_1 r_2} (v'_1, v'_2)} \\
\text{(C2)} \quad & \frac{v_1 \geq_{r_1} v'_1}{(v_1, v_2) \geq_{r_1 r_2} (v'_1, v'_2)} \\
\text{(A1)} \quad & \frac{\text{len } |v_2| > \text{len } |v_1|}{\text{Right } v_2 \geq_{r_1+r_2} \text{Left } v_1} \\
\text{(A2)} \quad & \frac{\text{len } |v_1| \geq \text{len } |v_2|}{\text{Left } v_1 \geq_{r_1+r_2} \text{Right } v_2} \\
\text{(A3)} \quad & \frac{v_2 \geq_{r_2} v'_2}{\text{Right } v_2 \geq_{r_1+r_2} \text{Right } v'_2} \\
\text{(A4)} \quad & \frac{v_1 \geq_{r_1} v'_1}{\text{Left } v_1 \geq_{r_1+r_2} \text{Left } v'_1} \\
\text{(K1)} \quad & \frac{|v : vs| = \varepsilon}{[] \geq_{r^*} v : vs} \\
\text{(K2)} \quad & \frac{|v : vs| \neq \varepsilon}{v : vs \geq_{r^*} []} \\
\text{(K3)} \quad & \frac{v_1 \geq_r v_2}{v_1 : vs_1 \geq_{r^*} v_2 : vs_2} \\
\text{(K4)} \quad & \frac{v_1 = v_2 \quad vs_1 \geq_{r^*} vs_2}{v_1 : vs_1 \geq_{r^*} v_2 : vs_2}
\end{aligned}$$

Congruence rules (C1), (C2) propagate a componentwise comparison through pairs: two concatenated parse trees compare by comparing each component in turn, with equality on the first component falling through to the second (C1). In place of Greedy’s unconditional “left beats right”, (A1) and (A2) compare by *length* first: whichever alternative produces the longer match wins outright (A1), and only when both sides match the same length does the left alternative win (A2) – recovering left-preference as a tie-breaker rather than an absolute rule. (A3), (A4) recurse structurally once a side has already been fixed. The remaining rules govern Kleene star lists: (K1) prefers the empty list $[]$ over any list of iterations that together still flatten to ε , and (K2) is its mirror image, preferring any non-empty-flattening list over an empty-flattening one; (K3), (K4) recurse into the list

structurally once the head elements are compared, first by the ordering on the star's body (r , via (K3)) and, only on equality, by the tail (via (K4)).

POSIX applied to the running example. Return once more to $r_1 = (a + ab)(b + \varepsilon)$ against $\mathbf{ab}$, comparing $v = (\text{Right}(a, b), \text{Right}())$ against $v' = (\text{Left } a, \text{Left } b)$ from Section 4.3. Both share the same regular expression, so (C2) applies to the first component: comparing $\text{Right}(a, b)$ against $\text{Left } a$ within $a + ab$ is a direct instance of (A1), since $\text{len } |(a, b)| = 2 > 1 = \text{len } |a|$. Hence $\text{Right}(a, b) \geq_{a+ab} \text{Left } a$, and by (C2), $v \geq_{r_1} v'$ – POSIX selects v , the *opposite* of Greedy's answer for the same $(r_1, \mathbf{ab})$ found in Section 4.3.1.

POSIX applied to the star example. For $r_2 = (a + b + ab)^*$ against $\mathbf{ab}$, comparing $v = [\text{Right } \text{Right}(a, b)]$ against $v' = [\text{Left } a, \text{Right } \text{Left } b]$ starts with (K3), comparing head elements $\text{Right } \text{Right}(a, b)$ and $\text{Left } a$ within the star's body $a + (b + ab)$. This is again (A1): $\text{len } |\text{Right}(a, b)| = 2 > 1 = \text{len } |a|$, so $\text{Right } \text{Right}(a, b) \geq_{a+(b+ab)} \text{Left } a$, and (K3) lifts this to $[\text{Right } \text{Right}(a, b)] \geq_{r_2} [\text{Left } a, \text{Right } \text{Left } b]$ directly – the single two-character iteration beats the two one-character iterations under POSIX.

POSIX applied to the infinite-chain examples. Rule (K1) resolves Section 4.3.2's pathology directly: for ε^* against ε , comparing $v_0 = []$ against $v_1 = [()]$ instantiates (K1) with $v = ()$, $vs = []$, whose flattening $|() : []| = |()| |[]| = \varepsilon \cdot \varepsilon = \varepsilon$ satisfies the rule's premise immediately, giving $[] \geq_{\varepsilon^*} [()]$ – and by the same argument against $v_2, v_3, \dots$, against every element of the chain at once, with no separate case analysis needed for each one. The same instantiation of (K1) applies to $(\varepsilon + a)^*$ against $\mathbf{a}$: every v_i in that chain other than $v_0 = [\text{Right } a]$ itself flattens to $\mathbf{a}$ once the trailing $\text{Right } a$ is accounted for, but the comparison that matters is between v_0 and the empty-flattening prefix chains layered in front of it in $v_1, v_2, \dots$ – and (K1) again picks the shorter, un-padded v_0 over any of them.

Unlike the Greedy order, $\geq_r$ is total and has a maximal element for *every* expression r , including the problematic ones of Section 4.3.2 – (K1) already picks out $[]$ as maximal for ε^* with no separate case analysis required [Sulzmann and Lu, 2014]. This single well-definedness property is what makes POSIX matching amenable to a direct algorithmic construction rather than a family of special cases, and it is exactly what Chapter 5 builds a parser around: a way to compute the $\geq_r$ -maximal parse tree without materializing and comparing every candidate.

Derivative-Based Parser

Chapter 4 fixed what is being computed – the $\geq_r$ -maximal parse tree of Section 4.3.3 – without saying how. This chapter presents Sulzmann and Lu’s answer [Sulzmann and Lu, 2014]: a parser built directly on the Brzozowski derivative of Section 3.2, in two forms. The first, direct construction computes a derivative sequence and then explicitly rebuilds a parse tree from it in a second, backward pass; the second, bit-coded construction fuses parse-tree bookkeeping into the derivative step itself, needing only one forward pass. Both are POSIX – they compute the identical $\geq_r$ -maximal tree, verified by direct comparison in Chapter 8 – and differ only in how that tree is produced. Both are developed here as theory first (Sections 5.1-5.4), then as the two Rust implementations that realize them, `deriv_std` (Section 5.5) and `deriv_bc` (Section 5.6). Chapter 6 repeats the exercise once more, replacing the Brzozowski derivative throughout with Antimirov’s partial derivative from Section 3.3 – and, as that chapter shows, changing more than just which derivative operator is used.

5.1 Core Algorithm

The direct construction follows the same two-phase shape informally sketched for Brzozowski’s derivative in Section 3.2: consume the input by taking successive derivatives, then decide whether the result is nullable. What is new here is what happens *between* those two phases – instead of a single Boolean, a parse tree is built. The construction is best understood in two halves: a forward pass that is nothing more than the derivative sequence already introduced, and a backward pass that walks that sequence in reverse to assemble a tree one consumed character at a time.

Forward pass. For input $x_1x_2\cdots x_n$, compute $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} r_n$ exactly as before, where $r_0 = r$ and $r_i = r_{i-1} \setminus x_i$. If r_n is not nullable, no parse tree exists and parsing fails. This half of the construction is identical to `match_deriv` (Section 3.2) with one difference: no simplification is applied between steps. `simplify` is safe for a Boolean matcher, since it only ever normalizes r_i to a language-equivalent expression, but the backward pass below needs to walk back through the *exact* sequence of derivative transformations that produced r_n from r_0 – a simplified r_i may no longer be the expression `inject` expects to see at that step.

Backward pass. Once r_n is known to be nullable, the parse tree v_n for the empty remainder is built directly, by a function `mkEps` defined by structural recursion on a

nullable expression:

$$\begin{aligned}
mkEps_\varepsilon &= () & mkEps_{r^*} &= [] \\
mkEps_{r_1 r_2} &= (mkEps_{r_1}, mkEps_{r_2}) & mkEps_{r_1+r_2} &= \begin{cases} \textit{Left } mkEps_{r_1} & \varepsilon \in L(r_1) \\ \textit{Right } mkEps_{r_2} & \text{otherwise} \end{cases}
\end{aligned}$$

$mkEps$ has no case for a literal x or for $\emptyset$, since neither is ever nullable – the function is only ever called on an expression already known to satisfy $n(r)$. Its only interesting case is alternation: exactly one of rule (A1)/(A2) of Section 4.3.3 is relevant here, since both sides of an empty match have length 0 – (A2)’s tie-break, "prefer left when lengths are equal", degenerates at the empty word to "prefer left whenever left is even possible", which is exactly the $\varepsilon \in L(r_1)$ guard above.

With v_n in hand, the remaining task is to turn it back into a parse tree of the *original* r_0 , one derivative step in reverse at a time. This is the job of inj , an injection function that takes a parse tree of a derivative $r \setminus x$ and re-inserts the consumed character x to produce a parse tree of r itself:

$$\begin{aligned}
inj_{r^* \setminus x}(v, vs) &= (inj_{r \setminus x} v) : vs \\
inj_{(r_1 r_2) \setminus x}(v) &= \begin{cases} (inj_{r_1 \setminus x} v_1, v_2) & v = (v_1, v_2), \text{ not nullable}(r_1) \\ (inj_{r_1 \setminus x} v_1, v_2) & v = \textit{Left}(v_1, v_2), \text{ nullable}(r_1) \\ (mkEps_{r_1}, inj_{r_2 \setminus x} v_2) & v = \textit{Right } v_2, \text{ nullable}(r_1) \end{cases} \\
inj_{(r_1+r_2) \setminus x}(v) &= \begin{cases} \textit{Left } (inj_{r_1 \setminus x} v_1) & v = \textit{Left } v_1 \\ \textit{Right } (inj_{r_2 \setminus x} v_2) & v = \textit{Right } v_2 \end{cases} \\
inj_{x \setminus x}() &= x
\end{aligned}$$

The three-way $(r_1 r_2)$ case is worth reading against the derivative rule it inverts (Section 3.2): $(r_1 r_2) \setminus x$ produced either $(r_1 \setminus x) r_2$ alone (when r_1 is not nullable) or that same term joined with $r_2 \setminus x$ via $+$ (when it is), tagged *Left/Right* respectively by the *Alt* this new $+$ introduces. inj undoes exactly that tagging: a bare pair means r_1 was not nullable and the derivative went straight into it; a *Left*-tagged pair means it was nullable but the derivative still chose to continue matching r_1 ; a *Right*-tagged value means the derivative instead jumped straight into r_2 , so r_1 ’s own contribution to the reconstructed pair has to be synthesized fresh via $mkEps_{r_1}$, since no part of it was consumed by x at all.

Parsing is then this two-phase process packaged into one function, applying $mkEps$ once at the end of the forward pass and inj once per character on the way back:

$$\textit{parse}(r, \varepsilon) = mkEps_r \quad (\text{if } \varepsilon \in L(r)) \quad \textit{parse}(r, xw) = inj_{r \setminus x}(\textit{parse}(r \setminus x, w))$$

Worked example. Return to $r_1 = (a + ab)(b + \varepsilon)$ against **ab**, whose two candidate parse trees were already established in Section 4.3. The forward pass computes $r_1 \xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) \xrightarrow{b} (\phi + (\phi b + \varepsilon))(b + \varepsilon) + (\varepsilon + \phi)$, nullable via its right disjunct $(\varepsilon + \phi)$. The backward pass starts from $mkEps$ of that final expression – $\textit{Right}(\textit{Right}(), \textit{Right}())$, abbreviated $(\textit{Right}(), \textit{Right}())$ once the outermost tag (there is only one alternative left at the top level once ϕ -branches are excluded) is elided – then injects **b** and finally **a** back in, arriving at $(\textit{Right}(a, b), \textit{Right}())$: the *longer* ab branch, matching the POSIX answer already claimed in Section 4.3.3 rather than Greedy’s $(\textit{Left } a, \textit{Left } b)$.

5.2 Correctness Properties

Sulzmann and Lu prove three results establishing that *parse* computes exactly the $\geq_r$ -maximal tree of Definition 4.3.3, not merely a valid one [Sulzmann and Lu, 2014]. This section states them; full proofs are in the reference and are not reproduced here, but the intuition behind each is worth making explicit, since it is exactly what justifies trusting the Rust implementation of Section 5.5 beyond "it agrees with the reference on every test case we tried" (Chapter 8 verifies that too, but the proofs are why agreement on finitely many test cases is expected to generalize).

Lemma 1 (Empty POSIX Parse Tree). For r with $\varepsilon \in L(r)$, $mkEps_r$ is the POSIX parse tree of r for the empty word. This grounds the base case of the induction the other two results build on: at $mkEps$'s only interesting case, alternation, both sides can only ever match length 0 (there is no remaining input to consume), so (A2)'s length comparison always ties and reduces to left-preference – exactly the $\varepsilon \in L(r_1)$ test $mkEps$ performs.

Lemma 2 (POSIX Preservation under Injection). If v is the POSIX parse tree of $r \setminus x$, then $inj_{r \setminus x}(v)$ is the POSIX parse tree of r . This is the result doing the real work, and its proof is correspondingly the most involved of the three. The informal argument: the derivative operation always tries r_1 before falling through to r_2 at a concatenation or alternation node (Section 3.2's $(r_1 r_2) \setminus x$ case only reaches $s \setminus x$ when r_1 is nullable, and even then keeps the r_1 -continuation branch as the left disjunct of the resulting $+$); inj reconstructing that exact same left-first bias at every backward step, rather than an arbitrary one, is what carries POSIX-maximality from the smaller expression $r \setminus x$ up to the original r .

Theorem 1 (POSIX Parsing). *parse* computes POSIX parse trees. This follows from Lemmas 1 and 2 by induction on the input word: the base case is Lemma 1 applied to r_n , and each inductive step is one application of Lemma 2, carrying POSIX-maximality from $r \setminus x_1 \cdots x_n$ back up through n injections to r itself.

5.3 Bit-Coded Optimization (Theory)

The direct construction of Section 5.1 must retain the entire forward-pass sequence $r_0, r_1, \dots, r_n$ (or, in an implementation, an equivalent call stack) so that inj has each $r_{i-1} \setminus x_i$ available to reconstruct against on the way back – the backward pass cannot begin until the forward pass has fully finished. Sulzmann and Lu's bit-coded construction (their Section 3.2, "Incremental Bit-Coded Forward Parse Tree Construction") [Sulzmann and Lu, 2014] removes this requirement by incrementally building the parse tree *during* the forward pass itself, so that by the time the last character is consumed, the winning tree's information is already sitting in the final expression – no second pass required.

Bit-code representation. Rather than a `ParseTree` value, a parse tree is represented as a flat sequence of bits: 0 marks a *Left* choice, 1 a *Right* choice, and inside a Kleene star, 0 marks "one more iteration" and 1 marks "stop". A pair of mutually recursive functions $encode_r/decode_r$ convert between a $\vdash v : r$ parse tree and its bit-string representation, structurally straightforward given the tagging already present in v itself.

Annotated regular expressions. The construction's core idea is to annotate the derivative operation's own *input*, not just its output, with partial bit-code information, so that by the time a sub-expression becomes ε its accumulated annotation already *is* the relevant slice of the final bit-string. An annotated regular expression ri extends plain r with a bit-list bs attached at every node except $\emptyset$ (which denotes no language and so has no parse tree to annotate):

$$ri ::= \emptyset \mid (bs@ \varepsilon) \mid (bs@ x) \mid (bs@ ri_1 \oplus ri_2) \mid (bs@ ri_1 ri_2) \mid (bs@ ri^*)$$

Two operations connect plain r to this annotated world. *internalize* converts r into ri with every bit-list initially empty, except at *Alt*: the left operand of $r_1 + r_2$ is tagged with a leading 0 and the right with a leading 1 at construction time, so that which alternative a sub-tree came from can always be recovered from its own annotation without re-inspecting the surrounding structure. *fuse*(bs, ri) prepends a bit-list onto whatever annotation ri 's own top-most node already carries – the one primitive every other operation in this construction is built from.

The refined derivative. The ordinary derivative operation is lifted to annotated expressions, written $ri \backslash_b l$, following the same case structure as $r \backslash l$ (Section 3.2) but threading bits through instead of discarding the annotation:

$$\begin{aligned} (bs@ ri_1 \oplus ri_2) \backslash_b l &= (bs@ (ri_1 \backslash_b l) \oplus (ri_2 \backslash_b l)) \\ (bs@ ri_1 ri_2) \backslash_b l &= \begin{cases} (bs@ (ri_1 \backslash_b l) ri_2) \oplus \text{fuse}(mkEpsBC_{ri_1}, ri_2 \backslash_b l) & \varepsilon \in L(ri_1) \\ (bs@ (ri_1 \backslash_b l) ri_2) & \text{otherwise} \end{cases} \\ (bs@ ri^*) \backslash_b l &= (bs@ \text{fuse}([0], ri \backslash_b l) ([]@ ri^*)) \end{aligned}$$

The concatenation case mirrors $(r_1 r_2) \backslash x$'s two-way split exactly, with one addition: on the branch where r_1 's derivative is abandoned in favor of jumping into r_2 , the bits r_1 would have contributed to an empty match ($mkEpsBC_{ri_1}$, below) are fused directly onto r_2 's derivative rather than discarded – this is the incremental step that replaces the direct construction's *later* call to $mkEps_{r_1}$ inside *inj* (Section 5.1) with work done immediately, during the forward pass. The star case attaches a leading 0 bit ("one more iteration") each time the body is derived once more; the outer bs stays where it is, on the new alternation/concatenation node the derivative rule produces, exactly as it does for plain *Alt*.

mkEpsBC. Mirroring *mkEps* (Section 5.1) but returning accumulated bits instead of a tree:

$$\begin{aligned} mkEpsBC_{(bs@ \varepsilon)} &= bs \\ mkEpsBC_{(bs@ ri_1 \oplus ri_2)} &= bs \mathrel{++} \begin{cases} mkEpsBC_{ri_1} & \varepsilon \in L(ri_1) \\ mkEpsBC_{ri_2} & \text{otherwise} \end{cases} \\ mkEpsBC_{(bs@ ri_1 ri_2)} &= bs \mathrel{++} mkEpsBC_{ri_1} \mathrel{++} mkEpsBC_{ri_2} \\ mkEpsBC_{(bs@ ri^*)} &= bs \mathrel{++} [1] \end{aligned}$$

Once the forward pass ends on a nullable annotated expression, *mkEpsBC* extracts its complete bit-string directly, and *decode_r* (using the *original*, un-annotated r – *decode*

needs to know the shape it is decoding bits *for*, which the annotated ri alone does not by itself determine (once its own annotations have been consumed) turns that bit-string into an ordinary parse tree. No second, backward pass over the input is needed at all.

Worked example. Run Section 5.1's $r_1 = (a + ab)(b + \varepsilon)$ against `ab` through this construction instead, to see the same POSIX answer assembled incrementally rather than reconstructed afterward (this trace is exactly what `cargo run - parse "(a|ab)(b|ε)" "ab" -parser deriv_bc -diag 3` prints; the tool's own `false/true` bits are written 0/1 below to match this section's convention). `internalize(r1)` tags the two *Alt* nodes' operands with 0/1 and leaves every other bit-list empty:

$$ri_0 = []@ \left([]@(0@a \oplus 1@([]@a \cdot []@b)) \cdot []@(0@b \oplus 1@ε) \right)$$

Deriving with respect to **a**: the left *Seq* operand $[]@(0@a \oplus 1@ab)$ is not nullable, so the *Seq* case's first branch applies, threading the derivative into it alone and leaving $(b + \varepsilon)$'s side untouched:

$$ri_1 = []@ \left([]@(0@ε \oplus 1@b) \cdot []@(0@b \oplus 1@ε) \right)$$

ri_1 's left operand is now nullable (via $0@ε$), so deriving with respect to **b** takes the *Seq* case's other branch: the left operand's own derivative continues on one side of a freshly introduced *Alt*, while *mkEpsBC* of that same left operand – $[0]$, the bit its nullable $0@ε$ disjunct contributes – is fused onto the right operand's derivative on the other:

$$ri_2 = []@ \left(1@([]@(0@b \oplus 1@ε)) \oplus [0, 0]@ε \right)$$

(the leading 1 tags "derivative continued into the left operand" as the *Right* alternative of this new node, since unlike *inj* – which only discovers this tagging when reconstructing backward – *deriv_bc* must commit to it immediately as part of the forward pass itself; the second 0 in $[0, 0]$ is $(b + \varepsilon)$'s own not-yet-derived left-disjunct tag, fused after ri_1 's $[0]$). ri_2 's top-level *Alt* has *both* disjuncts nullable here – $[0, 0]@ε$ trivially, and $1@([]@(0@b \oplus 1@ε))$ as well, via its own nested right disjunct $1@ε$ – so *mkEpsBC*'s *Alt* rule, which (mirroring *mkEps*'s own *Left*-preference from Section 5.1) always takes the left disjunct when it is nullable, regardless of the right disjunct also qualifying, picks $1@(\dots)$ over the shorter $[0, 0]@ε$:

$$mkEpsBC_{ri_2} = [] ++ mkEpsBC_{1@([]@(0@b \oplus 1@ε))} = 1 ++ ([] ++ mkEpsBC_{1@ε}) = 1 ++ [1] = [1, 1]$$

the same left-preference that makes r_1 's plain construction choose *Right* at the top level in Section 5.1's worked example: both encode "prefer continuing the match already in progress over falling back to the shorter alternative," just applied at different points – there, during *inj*'s backward pass; here, directly inside *mkEpsBC*'s forward-accumulated bits. Decoding against the original r_1 :

$$decode_{r_1}([1, 1]) = (Right(a, b), Right())$$

– the same tree Section 5.1's worked example reached via its explicit backward pass, this time read directly off the bits accumulated during the forward pass alone.

On simplification between steps. Section 5.1 noted that the plain construction's forward pass must *not* simplify between steps, since the backward pass needs to walk back

through the exact sequence of derivatives that produced it. The bit-coded construction has no such restriction, and `parse_bitcoded_recursive/parse_bitcoded_loop` (`src/parsers/deriv_bc/parse.rs`) do call `simp` after every step, exactly as this worked example's derivation does. This is safe here for the same reason `fuse` is: the information a backward pass would otherwise need to walk back for is already carried *on* the annotated expression itself, as bits attached to its nodes, so simplifying ri_i down to a smaller but bit-equivalent expression loses nothing *mkEpsBC* needs later – unlike the plain construction, where simplifying ri would destroy the very structure *inject* needs to reconstruct against.

5.4 Simplification Rules

Section 3.2's worked example already showed plain derivatives growing an extra $+$ at every nullable-prefix step with nothing to bound that growth; the bit-coded construction inherits the same problem; [Sulzmann and Lu, 2014] give the concrete case $a^* \xrightarrow{a} \varepsilon a^* \xrightarrow{a} \phi a^* + \varepsilon a^* \xrightarrow{a} (\phi a^* + \varepsilon a^*) + (\phi a^* + \varepsilon a^*) \xrightarrow{a} \dots$, where every step after the first is language-equivalent to εa^* but syntactically doubles in size. Left unchecked, this defeats the whole point of the bit-coded construction's incremental, single-pass promise: an unboundedly growing ri makes each subsequent $\backslash_b$ step correspondingly more expensive.

A simplification function *simp*, applied to a fixpoint after every derivative step, keeps the annotated expression's size bounded. Its rules – Phi-propagation through *Seq/Alt*, epsilon-elimination ($(bs@(bs'@\varepsilon) ri) \rightarrow fuse(bs ++ bs', ri)$, folding the eliminated node's own bits forward rather than discarding them), and flattening/deduplicating nested alternations, keeping the first occurrence of a syntactic duplicate – are all instances of the familiar algebraic identities from Section 3.2 ($\emptyset \cdot r = \emptyset$, $\varepsilon \cdot r = r$, $\emptyset + r = r$, $r + r = r$), applied here to the annotated grammar and combined with an explicit *isPhi* test that recognizes "empty language" expressions built up from these rules rather than only the literal $\emptyset$ constructor. Applied to the worked example above, *simp* would collapse every step back down to (a bit-annotated form of) εa^* , exactly as ordinary `simplify` does for the un-annotated case.

Sulzmann and Lu claim, but do not fully prove, that the resulting construction runs in time linear in the input, on the grounds that each of $\backslash_b$, *simp*, *fuse*, *mkEpsBC*, and *isPhi* only ever recurses over sub-calls bounded by the (now-simplified) expression's size, and note explicitly that detailed space-complexity estimates are left as future work [Sulzmann and Lu, 2014]. This thesis does not attempt to close that gap either; Chapter 8's benchmarks measure the implementation's actual behavior empirically rather than establishing a tighter asymptotic bound than the reference itself provides.

5.5 Rust Implementation: The `deriv_std` Parser

`deriv_std` (`src/parsers/deriv_std/`) is the Rust realization of Section 5.1's direct construction. It exposes two entry points computing the identical result by two different means – `parse_recursive` and `parse_loop` – discussed together in Section 5.5.4 below, after the shared machinery both call into.

5.5.1 ParseTree and flatten

The `ParseTree` type and its `flatten` function were already introduced in Section 4.2 (`src/types/tree.rs`) – `Empty`, `Char`, `Pair`, `Left`, `Right`, `Star` corresponding one to one with `()`, a literal, a pair, the two tagged injections, and a Kleene-star list. Nothing about the type itself is specific to `deriv_std`: it is the one representation every parser in this and the following chapter ultimately produces, precisely so that the agreement tests of Chapter 8 can compare their outputs directly rather than through some intermediate translation.

5.5.2 mk_eps

`mk_eps` (`src/regex/mk_eps.rs`) transcribes *mkEps* case for case: `Eps` to `Empty`, `Star` to an empty `Vec`, `Seq` to a `Pair` of the two recursive results, and `Alt` to `Left/Right` depending on `nullable(r1)` exactly as the theory prescribes. Called on `Lit` or `Phi` – which *mkEps* has no case for, since neither is ever nullable – it panics rather than silently returning a meaningless value; every call site only ever reaches `mk_eps` once nullability of its argument has already been established, so the panic is an assertion of an invariant the surrounding code maintains, not a reachable runtime error. This function lives in `regex/`, flat and outside `deriv_std/` specifically, because Chapter 6's `pderiv_std` needs the identical function for the identical reason – both build a `ParseTree` directly rather than bit-codes, and both need the same "parse tree for a nullable expression matched against nothing" primitive to do it.

5.5.3 inject

`inject` (`src/parsers/deriv_std/inject.rs`) is the direct transcription of *inj*. Its `Seq` case is the one worth reading closely against the theory: when `r1` is not nullable, only a `Pair` is ever handed in, and injection recurses into its first component; when `r1` is nullable, the incoming value is either `Left(Pair(v1, v2))` (the "continue `r1`" branch the derivative rule produces) or `Right(v2)` (the "`r1` already matched empty" branch), and only the second case calls `mk_eps(r1)` to synthesize the otherwise-absent left component – exactly the three-way split of Section 5.1's $inj_{(r_1 r_2) \setminus x}$. Every case that receives a value shaped differently from what the corresponding derivative rule could have produced – a bare `Pair` where a nullable `r1` demands a tagged `Left/Right`, for instance – panics rather than guessing, since such a mismatch can only mean the caller passed a tree that did not actually come from `r`'s own derivative sequence.

5.5.4 parse_recursive vs. parse_loop

Both functions (`src/parsers/deriv_std/parse.rs`) compute the identical result – verified directly, not just by construction, via `recursive_traced_agrees_with_loop_traced` (Chapter 8) – by two different means of implementing the same two-phase algorithm. `parse_recursive` recurses to the end of the input first, then calls `inject` on the way back up the call stack as the recursion unwinds – `deriv`'s "leading character removed" and `inject`'s "leading character re-inserted" mirror each other directly across the recursion, needing no explicit intermediate storage for the forward-pass sequence at all, since the call stack is that sequence. `parse_loop` makes the same sequence explicit as a `Vec<Regex>`, built by an ordinary forward `for` loop, then walks that vector backward

with a second loop calling `inject` at each step. The two exist side by side because they make different resource trade-offs unrelated to the algorithm itself: `parse_recursive`'s call stack depth grows with the input length, and can overflow it on sufficiently long input, where `parse_loop`'s explicit `Vec` instead grows on the heap – Chapter 8 measures exactly where that difference starts to matter.

5.6 Rust Implementation: The Bit-Coded Version

`deriv_bc` (`src/parsers/deriv_bc/`) realizes Section 5.3's construction.

5.6.1 ARegex – Bit-Annotated Regex

`ARegex` (`src/types/aregex.rs`) mirrors the annotated grammar *ri* directly: every variant except `Phi` carries its own `Vec<bool>` bit-prefix alongside the same structural payload `Regex` already carries (a `char` for `Lit`, boxed sub-expressions for the recursive cases). This is the one point in the whole implementation where a plain `Regex` is not enough: `deriv_bc`'s entire purpose is threading partial parse-tree information *through* the tree as it is derived, and that requires a place on every node to put it, not only at the leaves.

5.6.2 internalize/fuse

Both in `src/parsers/deriv_bc/internalize.rs`, `internalize` converts a plain `Regex` into a freshly bit-annotated `ARegex` with every list empty, except `Alt`: the left branch is fused with a leading `false` and the right with a leading `true` at construction time, exactly as the theory's *internalize* does. `fuse(bs, ri)` prepends `bs` onto `ri`'s own top-level annotation and is the one primitive the rest of this section's functions are all built from – including the degenerate case `fuse(bs, Phi) = Phi`, since there is no annotation slot on `Phi` to prepend anything to.

5.6.3 deriv_bc

`src/parsers/deriv_bc/deriv.rs` transcribes $\backslash_b$ case for case. Its `Alt` case keeps the outer bit-list on the resulting `Alt` node itself, rather than pushing it into either recursive call – a detail that looks like an arbitrary implementation choice here but turns out to matter a great deal once partial derivatives are introduced in Chapter 6, where the same choice, applied to a construction that no longer has a single combined node to attach bits to, is shown to cause genuine performance problems rather than a merely cosmetic difference. Its `Seq` case implements the nullable- r_1 branch's two strands exactly as written above – continue r_1 's own derivative, or jump into r_2 's with r_1 's empty-match bits fused on – and its `Star` case attaches the leading "one more iteration" bit via `fuse(&[false], ...)` before rewrapping the untouched star body. `deriv_bc` is called in a loop, once per input character, inside `parse_bitcoded/parse_bitcoded_loop` (`src/parsers/deriv_bc/parse.rs`), with `simp` applied after every step – the Rust realization of Section 5.4's fixpoint simplification, in `src/parsers/deriv_bc/simplify.rs`, whose cases (`Phi` propagation, epsilon-elimination via `fuse`, and list-flattening/first-occurrence deduplication for nested alternations) match *simp*'s directly.

5.6.4 mk_eps_bc

`src/parsers/deriv_bc/mk_eps_bc.rs` extracts the accumulated bit-string from a nullable `ARegex`, structurally identical to `mk_eps` but returning bits instead of a tree: `Alt` prefers whichever branch is nullable (left checked first, matching `mk_eps`'s own left-preference), `Seq` concatenates both children's bits after its own prefix, and `Star` appends a final `true` marking "no further iterations" onto whatever bits the completed iterations already accumulated.

5.6.5 decode

`src/regex/decode.rs` turns the plain, *original* `Regex` plus a flat `Vec<bool>` bit-string back into an ordinary `ParseTree`: `Alt` consumes one leading bit to choose `Left/Right`, `Seq` decodes both children in sequence, and `Star` loops, consuming a `false` per iteration until a `true` ends it – the direct inverse of `encode`, which this implementation never needs to run itself, since nothing here starts from an already-built `ParseTree` needing to be re-encoded. `decode` takes only a plain `Regex`, not an `ARegex` – by the time it is called, the bit-string alone carries everything needed, decoded against the shape of the *un*-annotated original expression. This is exactly why `decode` lives in flat `regex/` rather than inside `deriv_bc/` specifically: Chapter 6's `pderiv_bc` produces the same `(Regex, bits)` pair by an entirely different route and needs the identical final step to turn it into a tree.

5.7 Worked Example: paper_r1 End to End

Section 4.2 introduced $r_1 = (a + ab)(b + \varepsilon)$ against `ab` (`paper_r1`) on paper, deriving by hand that the POSIX result is $v = (Right(a, b), Right())$. What follows is that same derivation at diagnostics level 3 – the full structural trace, not the summarized construction steps of level 2 – produced by running `deriv_std` and `deriv_bc` themselves (`cargo run - parse "(a|ab)(b|ε)" "ab" -diag 3`, and again with `-parser deriv_bc`) rather than reconstructed from the theory alone: every derivative rule application, every intermediate expression, and every nullability check this chapter's two constructions actually perform, character by character.

Success: `deriv_std`. The forward pass names each rule applied at each step – Section 5.1's own $\partial(r_1 \cdot r_2, x)$ case split, visibly taken once per character – followed by the backward pass (Section 5.5.2/5.5.3) naming each *mkEps*/*inject* call in turn:

```
=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Standard (recursive parser)
Regex:               (a|ab)(b|epsilon)
Input:               "ab"
Result:              MATCH
Policy:              POSIX
=====
```

TIMING

```
=====
```

```

Parse time:          0.011ms
Expressions stored:  3 (r0 -> r1 -> r2)
-----=====
FORWARD PASS (Derivatives)
-----=====
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))

Step 1: d(r0, 'a')
  rule: d(r1.r2, 'a') [nullable(r1)=false]
        = d(r1, 'a').r2
r1 = Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = true [OK] position 1 matched

Step 2: d(r1, 'b')
  rule: d(r1.r2, 'b') [nullable(r1)=true]
        = d(r1, 'b').r2 + d(r2, 'b')
r2 = Alt(Seq(Alt(Phi, Alt(Seq(Phi, Lit('b'))), Eps)), Alt(Lit('b'),
  Eps)), Alt(Eps, Phi))
  nullable(r2) = true [OK] position 2 matched

-----=====
NULLABILITY CHECK
-----=====
nullable(r2) = true -> proceed to backward pass
-----=====
BACKWARD PASS (mkEps + inject)
-----=====
mkEps(r2):
  mkEps(r1 + r2): nullable(r1)=true -> Left(mkEps(r1))
v2 = Left (Right Right (), Right ())

inject(r1, 'b', v2):
  inject(Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps)),
    'b', Left (Right Right (), Right ()))
  -> (Right (), b), Right ()
v1 = (Right (), b), Right ()

inject(r0, 'a', v1):
  inject(Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
    Eps)), 'a', (Right (), b), Right ()))
  -> (Right (a, b), Right ())
v0 = (Right (a, b), Right ())

-----=====
RESULT
-----=====
Parse tree:          (Right (a, b), Right ())
Flattened:           "ab"

```

```
-----=====
END OF REPORT
-----=====
```

Every intermediate r_1/r_2 this report names is printed in full, not just referenced by number – **Expressions stored:** 3 ($r_0 \rightarrow r_1 \rightarrow r_2$) is exactly the derivative sequence Section 5.1 builds, and the backward pass’s own $v_2/v_1/v_0$ trace $mkEps(r_2)$ through two **inject** calls back to the final tree – character-for-character the v Section 4.2 derived on paper, as Theorem 1 (Section 5.2) guarantees it must be.

Success: deriv_bc. The same input against the same r_1 , through the bit-coded construction of Section 5.3: **internalize** annotating the regex once, then each **deriv_bc** + **simp** step shown against the derivative-on-bits rule it applies, then **mkEpsBC** extracting bits and **decode** turning them back into a tree:

```
-----=====
REGEX ENGINE DEBUG REPORT
-----=====
```

```
Mode:                POSIX Bitcoded
Regex:               (a|ab)(b|epsilon)
Input:               "ab"
Result:              MATCH
Policy:              POSIX
```

```
-----=====
TIMING
-----=====
```

```
Parse time:          0.014ms
Derivative steps:    2
```

```
-----=====
INTERNALIZE
-----=====
```

```
internalize((a|ab)(b|epsilon))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
```

```
-----=====
FORWARD PASS (deriv_bc + simp)
-----=====
```

```
Step 1 (char: 'a', position 1)
  ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
    []@([false]@'b' (+) [true]@epsilon))
  deriv_bc(ri0, 'a'):
    rule: (bs@ri1.ri2) \ 'a' [nullable_bc(ri1)=false]
      = bs@(ri1\'a'.ri2)
  simp -> []@([]@([false]@epsilon (+) [true]@'b') . []@([false]@'b'
    (+) [true]@epsilon))
  ri1 = []@([]@([false]@epsilon (+) [true]@'b') . []@([false]@'b'
    (+) [true]@epsilon))
  nullable_bc(ri1) = true [OK] bits accumulating
```

```

Step 2 (char: 'b', position 2)
  ri1 = []@([]@([false]@epsilon (+) [true]@'b')) . []@([false]@'b'
    (+) [true]@epsilon))
  deriv_bc(ri1, 'b'):
    rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=true]
      = bs@((ri1\'b'.ri2) (+) (fuse (mkEpsBC ri1) (ri2\'b')))
  simp -> []@([true]@([false]@'b' (+) [true]@epsilon) (+) [false,
    false]@epsilon)
  ri2 = []@([true]@([false]@'b' (+) [true]@epsilon) (+) [false,
    false]@epsilon)
  nullable_bc(ri2) = true [OK] bits accumulating

-----=====
NULLABILITY CHECK
-----=====
nullable_bc(ri2) = true -> proceed to mkEpsBC + decode
-----=====
MKEEPSBC + DECODE
-----=====
mkEpsBC(ri2) = [11]

decode((a|ab)(b|epsilon), [11]):
  = (Right (a, b), Right ())
-----=====
RESULT
-----=====
Bits:                [11]
Parse tree:          (Right (a, b), Right ())
Flattened:           "ab"
-----=====
END OF REPORT
-----=====

```

Where `deriv_std`'s report names three plain `Regex` expressions and two `inject` calls, `deriv_bc`'s names two annotated `ri0 -> ri1 -> ri2` steps and a single `mkEpsBC/decode` pair – the same structural difference Sections 5.5 and 5.6 describe in Rust terms, now visible directly in each construction's own trace. The final bits `[11]` select *Right* at both alternation points – the second alternative of $a + ab$, then the second (empty) alternative of $b + \varepsilon$ – which `decode` reads back as the identical $(\textit{Right}(a, b), \textit{Right}())$ `deriv_std`'s separate backward pass also produced.

Failure: `deriv_std`. Running the same r_1 against `b` instead – a string not in $L(r_1)$, since every derivation of r_1 must begin by matching `a`:

```

-----=====
REGEX ENGINE DEBUG REPORT
-----=====
Mode:                POSIX Standard (recursive parser)

```

```

Regex:          (a|ab)(b|epsilon)
Input:          "b"
Result:         NO MATCH
Policy:         POSIX
-----=====
TIMING
-----=====
Parse time:      0.006ms
Expressions stored: 2 (r0 -> r1)
-----=====
FORWARD PASS (Derivatives)
-----=====
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))

Step 1: d(r0, 'b')
  rule: d(r1.r2, 'b') [nullable(r1)=false]
        = d(r1, 'b').r2
r1 = Seq(Alt(Phi, Seq(Phi, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = false [FAIL] position 1

-----=====
NULLABILITY CHECK
-----=====
nullable(r1) = false -> no parse tree exists for full input
-----=====
PARTIAL RECOVERY
-----=====
No prefix matched.
-----=====
ERROR SUMMARY
-----=====
Failure at position: 1 (1-indexed)
Found:               'b'
Expected:            'a'
  b
  ^
-----=====
END OF REPORT
-----=====

```

The single derivative step taken, $\partial(r_1 \cdot r_2, b)$ with r_1 not nullable, derives r_1 down to a Seq of two Phis – not nullable – so the report moves directly from the nullability check to partial recovery and an error summary, never reaching a backward pass at all.

Failure: deriv_bc. The bit-coded construction on the identical input:

```

-----=====
REGEX ENGINE DEBUG REPORT
-----=====

```

```

Mode:                POSIX Bitcoded
Regex:               (a|ab)(b|epsilon)
Input:               "b"
Result:              NO MATCH
Policy:              POSIX
-----=====
TIMING
-----=====
Parse time:          0.006ms
Derivative steps:    1
-----=====
INTERNALIZE
-----=====
internalize((a|ab)(b|epsilon))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
-----=====
FORWARD PASS (deriv_bc + simp)
-----=====
Step 1 (char: 'b', position 1)
  ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
  deriv_bc(ri0, 'b'):
    rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=false]
      = bs@(ri1\'b'.ri2)
  simp -> phi
  ri1 = phi
  nullable_bc(ri1) = false [FAIL]
-----=====
NULLABILITY CHECK
-----=====
nullable_bc(ri1) = false -> no full parse tree exists
-----=====
PARTIAL RECOVERY
-----=====
No prefix matched.
-----=====
ERROR SUMMARY
-----=====
Failure at position: 1 (1-indexed)
Found:               'b'
Expected:             'a'
  b
  ^
-----=====
END OF REPORT

```

-----=====

`simp` collapses the single derivative step all the way to `phi` rather than stopping at an unreduced `Seq` of `Phis` the way `deriv_std`'s unsimplified report does – consistent with Section 5.4's own $\phi \cdot r \equiv \phi$ rule firing immediately – but both constructions report the identical position, character, and expected-set in their final error summary, since the diagnostics layer's expected-set (Chapter 9's Section 9.3.3) is read directly off r_1 's first literal set at the point of failure, independently of which construction, or how much simplification, produced it.

Partial-Derivative-Based Parser

Chapter 5 built two parsers around Brzozowski’s derivative, both computing the identical POSIX $\geq_r$ -maximal tree of Section 4.3.3. This chapter repeats the exercise with Antimirov’s partial derivative from Section 3.3 in place of $r \backslash x$ throughout – but, as Section 6.1 shows first, repeating the construction with a different derivative operator does not repeat its disambiguation guarantee. Neither flops14-extended [Sulzmann and Lu, 2014] nor Sulzmann’s teaching-page Haskell reference [Sulzmann] defines a POSIX partial-derivative parser at all – the paper’s bit-coded construction (Chapter 5) is Brzozowski-derivative-only, and the teaching page’s own bit-coded partial-derivative parser, `pDerivBC`, is built around a dedup rule this chapter shows to be fundamentally Greedy rather than POSIX. This chapter therefore builds two *Greedy* parsers, `pderiv_bc` (Section 6.3, following `pDerivBC` directly) and `pderiv_std` (Section 6.4, this project’s own construction, built to give `pderiv_bc` the same kind of matched non-bit-coded counterpart Chapter 5’s `deriv_bc` has in `deriv_std`) – both verified to agree with each other exactly, and with the two POSIX parsers on membership alone, in Chapter 8.

6.1 Extending Derivatives to Partial Derivatives

Section 3.3 already defined $pd(r, x)$ as a *set* of residual expressions rather than one combined expression, and noted the NFA reading: each element of $pd(r, x)$ is one state of an implicit NFA the original r denotes, alive after reading x . Extending this to parsing means pairing each surviving element of that set with enough information to reconstruct a parse tree once a particular strand turns out to be the winning one at the end of the input – concretely, either an accumulated bit-string per strand (Section 6.2) or, in this project’s own extension, an accumulated tree-injection closure per strand (Section 6.4). Either way, the algorithm shape is the same: maintain a *frontier* of `(residual, extra)` pairs, one per live strand, replace it at each character with the union of pd applied to every current strand (each contributing its own possibly-empty set of successor strands), and at the end of the input select the first strand in the frontier whose residual is nullable.

Why the result is Greedy, not POSIX. This last step – “the *first* nullable strand in the frontier” – is where this construction’s disambiguation policy is actually decided, and it is decided implicitly, as a consequence of how the frontier’s growth is kept finite rather than by any explicit ordering rule analogous to Section 4.3.3’s (A1)-(A4)/(K1)-(K4). Antimirov’s finiteness theorem (Section 3.3) guarantees the set of *distinct* partial derivatives is bounded, but nothing about the extended, tree-or-bit-carrying construction guarantees strands stay distinct as they accumulate – two different derivation paths can easily reach the identical residual *shape* by two different routes, each carrying its own bits

or injection. Kept without pruning, this set would multiply extra copies of an already-fully-explored residual instead of the bounded, syntactic set pd itself guarantees; pruned by comparing the accumulated bits or trees for POSIX-style "longest match wins" the way Section 4.3.3's ordering does, the resulting comparison would need to look inside possibly-incomplete, still-growing accumulated state rather than a finished `ParseTree`, which the ordering relation of Definition 4.3.3 is not defined over. The Haskell reference's actual choice – `nub2`, deduplicating purely on residual *shape* and keeping whichever strand reached that shape *first* – sidesteps this entirely by discarding the comparison altogether: once a residual shape has been claimed by some strand, every later strand reaching the identical shape is simply dropped, regardless of what either one's accumulated bits or tree would eventually have produced. "First" here means first in the list-ordered traversal pd 's own case structure produces, which explores the left operand of $+$ before the right at every step – so "first claim wins" is exactly Greedy's leftmost-commits-immediately rule (Section 4.3.1), not POSIX's longest-wins-with-left-as-tiebreak rule. This reading is this thesis's own account of why the construction lands on Greedy rather than POSIX; neither primary source states it in these terms; both parsers built in this chapter are verified empirically in Chapter 8 to compute exactly the Greedy answer wherever it is known to differ from POSIX (the `paper_r1`/`paper_r2` examples of Section 4.3), which is the strongest evidence available for it in the absence of a reference construction to check against directly.

6.2 Bit-Coded Variant

Sulzmann's teaching-page Haskell transcript gives a bit-coded partial-derivative construction, `pDerivBC`, that this thesis's `pderiv_bc` follows directly [Sulzmann]. It extends pd with per-strand bits the same way Section 5.3's $\backslash_b$ extends the Brzowski derivative – but, notably, keeps those bits *external* to the regular-expression tree, as a plain list carried alongside a plain, un-annotated r , rather than folding them into an `ARegex`-shaped tree the way $\backslash_b$ does:

$$\begin{aligned}
pDerivBC(x, \varepsilon) &= \{\} \\
pDerivBC(x, y) &= \begin{cases} \{(\varepsilon, [])\} & x = y \\ \{\} & x \neq y \end{cases} \\
pDerivBC(x, r + s) &= nub2(\{(r', 0:bs) \mid (r', bs) \in pDerivBC(x, r)\} \cup \{(s', 1:bs) \mid (s', bs) \in pDerivBC(x, s)\}) \\
pDerivBC(x, rs) &= \begin{cases} nub2(\{(rs', bs) \mid (r', bs) \in pDerivBC(x, r), rs' = r's\} \\ \quad \cup \{(s', mkEpsBC_r ++ bs) \mid (s', bs) \in pDerivBC(x, s)\}) & n(r) \\ \{(smartC(r', s), bs) \mid (r', bs) \in pDerivBC(x, r)\} & \text{otherwise} \end{cases} \\
pDerivBC(x, r^*) &= nub2(\{(smartC(r', r^*), 0:bs) \mid (r', bs) \in pDerivBC(x, r)\})
\end{aligned}$$

where $mkEpsBC_r$ is the plain- r analogue of Section 5.6.4's $mkEpsBC$ (identical structure, walking an un-annotated tree instead of an `ARegex`), and `nub2` deduplicates a list of (`residual`, `bits`) pairs purely by comparing the residual, keeping the first occurrence. The concatenation case's first strand deliberately keeps r' wrapped in a literal, non-smart-constructed $r's$ rather than $smartC$ – an asymmetry present in the reference itself, preserved here rather than "corrected", since correctness of this construction is judged against literal agreement with that reference (Chapter 8), not against an independently-derived ideal form.

A negative result: why not ARegex. Given Chapter 5’s bit-coded parser threads bits *through* the tree via ARegex, extending `pderiv_bc` the same way – so that it, too, could share `internalize/fuse/mkEpsBC/decode` with `deriv_bc` – looks like the natural, structurally consistent choice, and was built and tested as exactly that during this project’s development. It was correct: verified byte-for-byte identical to the external-bits construction above across a 20,000-case differential fuzzer. It was also, on real ambiguous input, dramatically slower – on $(a + (b + ab))^*$ against "ab" repeated, roughly 240 microseconds at 10 characters growing to roughly 14 seconds at 40, a blowup far beyond what the same pattern costs either `pderiv_std` or `pderiv_bc` as actually shipped. The diagnosed cause connects directly to Section 5.6.3’s observation that `deriv_bc`’s `Alt` case keeps its outer bit-list on the `Alt` node itself rather than pushing it into either branch: partial derivatives, unlike Brzozowski’s, *flatten* every alternation branch out into its own independent strand rather than keeping the `Alt` structure intact, so an ARegex-based `pderiv_bc` has no outer node left to attach bits to and must fuse them into each flattened branch instead. That fusion makes two strands that are semantically identical – reachable from the same residual, differing only in *which derivation path* produced them – come out syntactically different depending on that path, which silently defeats nub2’s shape-based deduplication: strands the un-annotated construction above would collapse into one instead survive as separate, and the frontier this is meant to keep bounded (Section 6.1) grows instead. Keeping bits external and structurally separate from the shape used for deduplication, as the teaching-page reference itself does, is what makes Antimirov’s finiteness guarantee actually hold in this implementation; ARegex was designed around Brzozowski’s single-threaded derivative, where exactly one combined tree exists to annotate at every step, and does not transfer cleanly to a construction that has already split into independent strands with nothing to annotate collectively at all.

6.3 Rust Implementation: The `pderiv_bc` Parser

`src/parsers/pderiv_bc/pderiv.rs` transcribes *pDerivBC* case for case, as `Vec<(Regex, Vec<bool>)>` rather than a set (order matters here, encoding Greedy priority directly in list position, so a `HashSet` would lose exactly the information the construction depends on). `mk_eps_bits` implements $mkEpsBC_r$; nub2 dedups on the residual `Regex` alone via a `HashSet<Regex>` used only to test membership, keeping the first-seen pair for each distinct residual and discarding the rest. `pderiv_bc`’s own `Seq` case matches the reference literally, down to the same asymmetric use of plain `Regex::seq` (not `smart_seq`) for its first strand.

`step_frontier/select_bits` (`src/parsers/pderiv_bc/parse.rs`) implement the frontier-driving loop of Section 6.1: at each character, `step_frontier` replaces every (`residual`, `bits`) pair in the current frontier with the union of `pderiv_bc` applied to it, concatenating each result’s bits onto that strand’s already-accumulated prefix; at the end of the input, `select_bits` scans the frontier in list order and returns the first nullable strand’s `bits ++ mk_eps_bits(residual)`. `parse_pderiv_bc` drives this loop to completion and passes the winning bit-string, together with the *original* `Regex` (never the derived residual), to `decode` (Section 5.6.5) – the same shared decoder Chapter 5’s `deriv_bc` uses, since both constructions end up at the identical (`Regex`, `bits`) shape by the time decoding is needed, whatever route each took to get there.

Worked example. Run Section 5.3's $r_1 = (a+ab)(b+\varepsilon)$ against `ab` through `pderiv_bc` instead of `deriv_bc` – same regex, same input, different parser, to see Greedy and POSIX visibly disagree on the same worked example rather than only being told they can. (Again, this is exactly `cargo run -p parse "(a|ab)(b|ε)" "ab" -parser pderiv_bc -diag 3`'s output.) The frontier starts as a single strand, the whole regex with no bits yet:

$$[(r_1, [])]$$

Deriving with respect to **a**: r_1 's left *Seq* operand is not nullable, so `pDerivBC`'s *Seq* case applies uniformly to the single strand, producing one residual per partial derivative of $(a+ab)$ with respect to **a** – two strands where the plain bit-coded construction kept one:

$$[(b+\varepsilon, [0]), (b \cdot (b+\varepsilon), [1])]$$

the first strand from a 's own derivative (ε , smart-concatenated away against $(b+\varepsilon)$), tagged 0 for having come from $(a+ab)$'s left disjunct; the second from ab 's derivative (b , concatenated onto $(b+\varepsilon)$), tagged 1 for the right disjunct. Deriving with respect to **b** steps *both* strands independently: strand 0's $(b+\varepsilon)$ splits again into two partial derivatives of its own (ε from the left disjunct, $\emptyset$ from the right, `nub2`-deduplicated and the $\emptyset$ one dropped), while strand 1's $b \cdot (b+\varepsilon)$ takes the *Seq* case's not-nullable branch and derives only its left factor:

$$[(\varepsilon, [0,0]), (b+\varepsilon, [1])]$$

Selection scans this final frontier in list order and takes the *first* nullable strand – strand 0, $(\varepsilon, [0,0])$, is checked and accepted before strand 1 is even inspected, regardless of strand 1 also being nullable. This is Greedy's leftmost-commits rule made mechanical: strand 0 is exactly the "matched **a** then **b** separately" reading, the first alternative `pDerivBC` generated back at the first derivative step, and it wins here purely because it appears first in the frontier – not because any length comparison favors it. Decoding its accumulated bits:

$$\text{decode}_{r_1}([0,0]) = (\text{Left } a, \text{Left } b)$$

the *shorter* **a+b** reading – the opposite answer from Section 5.3's $(\text{Right}(a,b), \text{Right}())$ on the identical regex and input. Nothing about r_1 or "`ab`" changed between the two traces; only the disambiguation policy governing which nullable strand/branch wins did, exactly the distinction Section 6.1 argues is built into `nub2`'s shape-only, first-occurrence-wins deduplication.

6.4 Rust Implementation: The `pderiv_std` Parser

Unlike every other parser in Chapters 5 and 6, `pderiv_std` transcribes no reference construction – neither primary source defines a partial-derivative parser that builds a `ParseTree` directly, without going through a bit-string intermediate. It exists to fill a structural gap the rest of this thesis's four-parser comparison would otherwise have: Chapter 5's `deriv_std` and `deriv_bc` form a matched pair sharing the same underlying derivative and differing only in how a successful derivation becomes a tree – a direct backward pass versus incremental bit annotation – which makes comparing them, both for correctness and for the performance analysis of Chapter 8, a controlled, apples-to-apples comparison. `pderiv_bc` (Section 6.3) had no such sibling before `pderiv_std` was built:

it was the only Greedy partial-derivative parser available, with nothing sharing its underlying derivative but differing only in tree-construction strategy to compare it against. `pderiv_std` closes that gap, giving all four parsers this thesis studies the same paired structure – Section 5 covering the derivative pair, Section 6.3 and this section covering the partial-derivative pair.

Injection closures in place of bits. `pDerivBC`'s bit-vector trick works because bits alone are enough for `decode` to later reconstruct a tree, given the *original* expression to decode them against. Building the tree directly during the forward pass, with no such second decoding step, needs something that can still be applied once a strand's eventual `mk_eps` value is known, without needing the original expression again – an injection function `ParseTree -> ParseTree` that wraps whatever value a residual eventually produces in exactly the `Left/Right/Pair` structure needed to have come from the *original* expression, not the residual. `type Inj = Rc<dyn Fn(ParseTree) -> ParseTree>` (`src/parsers/pderiv_std/inject.rs`) is this closure – `Rc` rather than `Box`, since a fan-out step such as `Alt` shares one parent strand's injection across more than one child strand, which a uniquely-owned closure could not do. Every frontier strand carries `(Regex, Inj)` in place of `pderiv_bc`'s `(Regex, Vec<bool>)` – the closure *is* this parser's bit-vector, composed at each step rather than concatenated.

`pderiv_tree` (`src/parsers/pderiv_std/pderiv_tree.rs`) mirrors `pderiv_bc`'s case structure directly, with injection-wrapping in place of bit-prepend at every case – a literal match wraps its result in a closure building `ParseTree::Char` directly (where the bit-coded version records no bits at all, since there was nothing to choose between); alternation's two branches wrap in `Left/Right`-building closures composed with each branch's own; concatenation's nullable-first-operand case builds `Pair(mk_eps(r1), ...)` directly for its "jump into `r2`" strand, calling the *shared* `mk_eps` (Section 5.5.2) eagerly rather than deferring it the way `mkEpsBC`'s bits would; and Kleene star's closure unpacks and prepends one iteration onto the existing `Star` list. `nub_tree` dedups exactly as `pderiv_bc`'s `nub2` does, on the residual alone, keeping the first (highest Greedy priority) strand's injection. `step_frontier/select` (`src/parsers/pderiv_std/parse.rs`) mirrors `pderiv_bc`'s `step_frontier/select_bits` one to one, composing injections instead of concatenating bits and applying the winning strand's closure directly to `mk_eps(residual)` at the end – no decoding step required, since the tree was under construction the entire time rather than encoded.

Because `pderiv_tree` shares `pderiv_bc`'s exact case structure, a strand-for-strand differential comparison between the two is close to free to state and check: Chapter 8's 20,000-case fuzzer finding the two implementations byte-identical on every generated regular expression and input, together with the shared `paper_r1/paper_r2` examples both compute the documented Greedy answer for, is the actual evidence available that this thesis's own partial-derivative-to-parse-tree extension is correct – in the absence of a reference construction, as Section 6.1 already noted, two independently-shaped implementations of the same specification agreeing exactly is the strongest correctness signal available.

6.5 Worked Example: paper_r1 End to End

Section 4.3 constructed, by hand, the *second* valid derivation of $r_1 = (a + ab)(b + \varepsilon)$ against `ab` (`paper_r1`) that Section 5.7 deliberately left aside: $v' = (\text{Left } a, \text{Left } b)$, matching the *first* alternative of $a + ab$ against `a` alone rather than the second alternative against `ab` in one step. This is the Greedy result – leftmost-alternative priority, decided locally at $a + ab$ without looking ahead at what $b + \varepsilon$ could still consume – and it is exactly what `pderiv_std` and `pderiv_bc` compute, not $v = (\text{Right}(a, b), \text{Right}())$ from Section 5.7. What follows is the full diagnostics level 3 trace for both – every partial derivative computed at every frontier step, not the summarized residual list level 2 prints – run against the identical r_1/ab (`cargo run - parse "(a|ab)(b|ε)" "ab" -parser pderiv_std -diag 3`, and again with `-parser pderiv_bc`): the frontier construction this chapter developed, actually computing the *other* tree Section 4.3 worked out by hand, and a concrete demonstration that this chapter’s parsers and Chapter 5’s are not interchangeable substitutes for the same problem: same input, same regular expression, two different, individually correct answers.

Success: `pderiv_std`. Each step shows every residual in the frontier *before* the step, the rule (Section 6.4’s `pderiv_tree`, one case per residual depending on whether its own head is nullable) applied to each, and the resulting frontier *after*:

```
-----=====
REGEX ENGINE DEBUG REPORT
-----=====
Mode:                Standard Partial-Derivative (pderiv_std)
Regex:               (a|ab)(b|epsilon)
Input:               "ab"
Result:              MATCH
Policy:              GREEDY
-----=====
TIMING
-----=====
Parse time:          0.018ms
Steps:               2
-----=====
INITIAL FRONTIER
-----=====
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
    Alt(Lit('b'), Eps))
-----=====
FORWARD PASS (pderiv_tree per residual, frontier flattened)
-----=====
Step 1 (char: 'a', position 1)
  frontier before (1 residual(s)):
    [0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
      Eps))
  pderiv_tree('a', .) applied to each:
    from [0]:
```

```

rule: pderiv_tree('a', r1.r2) [nullable(r1)=false]
    = [(r1'.r2, inj: continue r1) | (r1',_) <-
        pderiv_tree(c,r1)]
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not
    printable)
-> Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps)) (+ injection
    closure, not printable)
frontier after (2 residual(s)):
[0] Seq(Eps, Alt(Lit('b'), Eps))
[1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
a residual in this frontier is nullable [OK] parse tree
available

```

Step 2 (char: 'b', position 2)

```

frontier before (2 residual(s)):
[0] Seq(Eps, Alt(Lit('b'), Eps))
[1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
pderiv_tree('b', .) applied to each:
from [0]:
rule: pderiv_tree('b', r1.r2) [nullable(r1)=true]
    = nub_tree $ [(r1'.r2, inj: continue r1) | (r1',_) <-
        pderiv_tree(c,r1)]
        ++ [(r2', inj: mk_eps(r1) paired with r2's
            result) | (r2',_) <- pderiv_tree(c,r2)]
-> Eps (+ injection closure, not printable)
from [1]:
rule: pderiv_tree('b', r1.r2) [nullable(r1)=false]
    = [(r1'.r2, inj: continue r1) | (r1',_) <-
        pderiv_tree(c,r1)]
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not
    printable)
frontier after (2 residual(s)):
[0] Eps
[1] Seq(Eps, Alt(Lit('b'), Eps))
a residual in this frontier is nullable [OK] parse tree
available

```

```

-----=====
SELECTION (first nullable residual, priority/list order)
-----=====
any nullable residual in final frontier = true -> proceed to
    injection (already applied by select)
-----=====

```

RESULT

```

-----=====
Parse tree:          (Left a, Left b)
Flattened:           "ab"
-----=====

```

END OF REPORT

=====

Step 1's frontier already shows the fork this chapter's theory predicts: residual [0], `Seq(Eps, Alt(Lit('b'), Eps))`, is the strand that consumed `a` via `a+ab`'s *first* alternative; residual [1], nested one step deeper, is the strand that instead committed to `a+ab`'s *second* alternative and still owes a `b` before `b + ε` is even reached. `nub_tree` keeps both alive (they are structurally distinct residuals), and at step 2 the first-listed, first-priority residual is the one that reaches `Eps` – nullable – first: Greedy's leftmost-alternative priority, operating directly on frontier order rather than on any global comparison between the two candidate trees.

Success: `pderiv_bc`. The bit-coded sibling (Section 6.3) reaches the same tree via `nub2`-deduplicated (`Regex, bits`) pairs instead of (`Regex, Inj`) closures, each frontier entry carrying its accumulated bits alongside its residual:

=====

REGEX ENGINE DEBUG REPORT

=====

```
Mode:                Bitcoded Partial-Derivative (pDerivBC)
Regex:               (a|ab)(b|epsilon)
Input:               "ab"
Result:              MATCH
Policy:              GREEDY
```

=====

TIMING

=====

```
Parse time:          0.036ms
Steps:               2
```

=====

INITIAL FRONTIER

=====

```
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
```

=====

FORWARD PASS (pDerivBC per residual, frontier flattened)

=====

Step 1 (char: 'a', position 1)

frontier before (1 residual(s)):

```
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
  Eps))  bits-so-far = []
```

`pDerivBC('a', .)` applied to each:

from [0]:

```
rule: pDerivBC('a', r1.r2) [nullable(r1)=false]
      = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
-> (Alt(Lit('b'), Eps), [0])
-> (Seq(Lit('b'), Alt(Lit('b'), Eps)), [1])
```

frontier after (2 residual(s)):

```
[0] Alt(Lit('b'), Eps)  bits = [0]
```

```

    [1] Seq(Lit('b'), Alt(Lit('b'), Eps))    bits = [1]
a residual in this frontier is nullable [OK] bits accumulating

Step 2 (char: 'b', position 2)
frontier before (2 residual(s)):
    [0] Alt(Lit('b'), Eps)    bits-so-far = [0]
    [1] Seq(Lit('b'), Alt(Lit('b'), Eps))    bits-so-far = [1]
pDerivBC('b', .) applied to each:
from [0]:
    rule: pDerivBC('b', r1+r2) = tag(0, pDerivBC('b',r1)) ++
        tag(1, pDerivBC('b',r2)), nub2
    -> (Eps, [0])
from [1]:
    rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
        = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
    -> (Alt(Lit('b'), Eps), [])
frontier after (2 residual(s)):
    [0] Eps    bits = [00]
    [1] Alt(Lit('b'), Eps)    bits = [1]
a residual in this frontier is nullable [OK] bits accumulating

```

```

=====
SELECTION (first nullable residual, priority/list order)
=====
any nullable residual in final frontier = true -> proceed to
    mkEpsBC + decode
=====
MKEEPSBC + DECODE
=====
selected residual's accumulated bits ++ mkEpsBC(residual) = [00]

decode((a|ab)(b|epsilon), [00]):
    = (Left a, Left b)
=====
RESULT
=====
Bits:                [00]
Parse tree:          (Left a, Left b)
Flattened:           "ab"
=====
END OF REPORT
=====

```

Bits [00] select *Left* at both alternation points – exactly v' 's shape – and the winning residual at each step is identical to `pderiv_std`'s first-listed one above, consistent with `pderiv_tree` being built to mirror `pderiv_bc`'s case structure one to one (Section 6.4).

Failure: `pderiv_std`. Against **b** instead of **ab**: the first partial derivative of r_1 by **b** produces no residual at all, since neither alternative of $a + ab$ begins with **b**, so the frontier

is empty after step 1 and no selection ever runs:

```
=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                Standard Partial-Derivative (pderiv_std)
Regex:               (a|ab)(b|epsilon)
Input:               "b"
Result:              NO MATCH
Policy:              GREEDY
=====
TIMING
=====
Parse time:          0.005ms
Steps:               1
=====
INITIAL FRONTIER
=====
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
    Alt(Lit('b'), Eps))
=====
FORWARD PASS (pderiv_tree per residual, frontier flattened)
=====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
    [0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
      Eps))
  pderiv_tree('b', .) applied to each:
  from [0]:
    rule: pderiv_tree('b', r1.r2) [nullable(r1)=false]
      = [(r1'.r2, inj: continue r1) | (r1',_) <-
        pderiv_tree(c,r1)]
    -> []
  frontier after (0 residual(s)):
  no nullable residual in this frontier  [FAIL]

=====
SELECTION (first nullable residual, priority/list order)
=====
any nullable residual in final frontier = false -> no full parse
  tree exists
=====
PARTIAL RECOVERY
=====
No prefix matched.
=====
ERROR SUMMARY
=====
Failure at position:  1 (1-indexed)
```

```

Found:                'b'
Expected:             'a'
  b
  ^

```

```

-----=====
END OF REPORT
-----=====

```

Failure: pderiv_bc. The bit-coded construction on the identical input, reaching the same empty frontier by the same rule:

```

-----=====
REGEX ENGINE DEBUG REPORT
-----=====

```

```

Mode:                Bitcoded Partial-Derivative (pDerivBC)
Regex:              (a|ab)(b|epsilon)
Input:              "b"
Result:             NO MATCH
Policy:             GREEDY

```

```

-----=====
TIMING
-----=====

```

```

Parse time:          0.004ms
Steps:              1

```

```

-----=====
INITIAL FRONTIER
-----=====

```

```

[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))

```

```

-----=====
FORWARD PASS (pDerivBC per residual, frontier flattened)
-----=====

```

```

Step 1 (char: 'b', position 1)

```

```

  frontier before (1 residual(s)):

```

```

    [0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
      Eps))  bits-so-far = []

```

```

  pDerivBC('b', .) applied to each:

```

```

    from [0]:

```

```

      rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
          = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]

```

```

      -> []

```

```

  frontier after (0 residual(s)):

```

```

  no nullable residual in this frontier  [FAIL]

```

```

-----=====
SELECTION (first nullable residual, priority/list order)
-----=====

```

```

any nullable residual in final frontier = false -> no full parse
  tree exists

```

```

-----=====
PARTIAL RECOVERY
-----=====
No prefix matched.
-----=====
ERROR SUMMARY
-----=====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'
    b
    ^
-----=====
END OF REPORT
-----=====

```

The two chapters' constructions disagree, deliberately, on which tree a successful match produces – Greedy versus POSIX – but agree exactly on the boolean question of membership and on the diagnostics reported for a non-match, since neither disambiguation policy nor bit-versus-closure tree construction is ever reached until after the underlying derivative/partial-derivative walk has already found at least one live residual.

Frontend: Parsing External Regular Expression Syntax

Every example so far in this thesis has been written directly in `Regex` terms – the six-constructor core algebra of Section 2.3, exactly what `deriv/pderiv` operate on. No real regular-expression user writes that way: a Snort rule, a SpamAssassin filter, or an ordinary call to `cargo run - match` arrives as an ordinary PCRE-style pattern *string* – `"[a-c]+\d{2}"`, not `Seq(Star(...), ...)`. `src/frontend/` is the module that closes this gap: it parses that string into its own surface-syntax AST, then lowers that AST into the `Regex` every parser in Chapters 5 and 6 already understands. It is modeled throughout on `Text.Regex.PDeriv`'s own `ExtPattern/Parse/Translate` modules from `luzhuomi/regex-pderiv`, Kenny Lu's own reference implementation of [Sulzmann and Lu \[2012\]](#)'s partial-derivative submatching – a second, independent reference alongside [Sulzmann and Lu \[2014\]](#)'s own teaching-page material (Section 9.2.1), used here specifically because the `flops14`-extended paper itself has nothing to say about surface syntax: POSIX parsing is defined directly over the same six-constructor `Regex` algebra this thesis's core inherits, with no PCRE-facing frontend of its own.

This chapter is deliberately placed before Chapter 8 rather than folded into it: Section 8.3's real-world corpus depends on everything described here (surface-syntax parsing, desugaring, and the substring-search padding scheme) to turn raw Suricata/SpamAssassin/RegexLib rule text into `Regex` values in the first place, so the mechanism needs to be established before its results can be read.

7.1 Two ASTs, One Lowering Step

`src/frontend/` is built around a deliberate separation into two abstract syntax trees connected by a single function, rather than parsing pattern text directly into `Regex`:

$$\text{pattern string} \xrightarrow{\text{parse_ext_pattern}} \text{ExtPat} \xrightarrow{\text{translate}} \text{Regex}$$

`ExtPat` (`ext_pattern.rs`) is the *external*, surface-syntax AST: marking and non-marking groups, `{m,n}` bounds, `\d/\w/\s` shorthands, `^/$` anchors, and a `greedy: bool` flag on every quantifier (`a*` vs. `a*?`) – everything a PCRE-style string can express, kept exactly as written, with nothing desugared yet. `Regex` (Section 2.3) is the *internal*, six-constructor core every parser derives or partial-derives directly: `Phi` | `Eps` | `Lit` | `Seq` | `Alt` | `Star`, no groups, no bounds, no anchors, no laziness flag. Anything `ExtPat` can say that `Regex` cannot represent is desugared away by `translate` (Section 7.3) or, for `^/$` specifically, erased into `Eps` and handled one level up instead (Section 7.4).

Keeping `ExtPat` as an honest AST in its own right, rather than routing straight from parser to `Regex` in one pass, is what makes `translate` independently testable: `translate.rs`'s own unit tests (below) construct `ExtPat` values by hand and check their `Regex` output directly, with no dependency on `parse.rs`'s recursive-descent logic at all. It is the same separation-of-concerns argument Section 5.5.1 makes for keeping `ParseTree` construction and `flatten/decode` as distinct steps rather than one fused pass – here applied one layer further out, at the point where a PCRE-style string first becomes structured data at all.

7.2 Parsing Surface Syntax

`frontend/parse.rs` is an ordinary recursive-descent parser over a `Chars` iterator, structured directly after `Text.Regex.PDeriv.Parse`'s own grammar – `p_ere/p_branch/p_exp/p_atom/p_group/p_charclass/p_enum/p_bound` become `parse_or/parse_concat/parse_postfixed/parse_atom/parse_group/parse_charclass/parse_class_enum/parse_bound` one for one, each handling exactly the surface-syntax layer its reference counterpart does (alternation, then concatenation, then postfix quantifiers, then a single atom). Four deliberate departures from that reference are documented directly in the module's own header comment, each because real-world pattern text (Section 8.3's corpus, drawn from Suricata, SpamAssassin, and `RegexLib` rules) needs a decision the reference either doesn't make or makes differently:

- **Class shorthands are PCRE character classes, not literal escapes.** The reference's `Translate.hs` reads a bare `EEscape c` as the literal character `c` always; this frontend special-cases `d/D/w/W/s/S` into character classes (Section 7.3's `alphabet.rs`) instead, since real datasets use them that way throughout.
- **Backreferences and lookahead are rejected outright.** `\1-\9` and `(?=/(?!/(?<=/(?<!` produce a descriptive parse error rather than the reference's literal-digit or 3-digit octal-ASCII escape reading – both describe languages that are not regular in the formal sense this thesis's entire construction depends on, so silently mis-parsing them into *some* `Regex` would be a worse outcome than refusing them, and Section 8.3's extraction pipeline depends on this refusal being loud and categorized, not silent.
- **Named groups are accepted.** `(?<name>...)` and `(?P<name>...)` parse successfully, dropped to their unnamed form during `translate` – a name carries no meaning once lowered to `Regex`, since this project has no capture-group semantics at all, but rejecting a construct this common in real Suricata/`RegexLib` rules purely because the reference parser predates it would reject working patterns for no benefit.
- **`\xHH` hex escapes are accepted**, another addition absent from the reference, for the same reason.

`parse_bound` additionally accepts a lazy-quantifier marker (`a*?`, `a+?`, `a{2,4}?`) and records it as `ExtPat`'s **greedy**: `bool` flag – correctly read, but, as Section 7.3 explains, not honored by `translate`. Grouping, alternation, and concatenation otherwise follow the reference's precedence exactly: alternation binds loosest, then concatenation, then postfix quantifiers, then a single atom (literal, escape, character class, group, or anchor).

7.3 Translating ExtPat into Regex

`translate` (`frontend/translate.rs`) is a single structural recursion over `ExtPat`, one match arm per constructor, each either a direct one-to-one mapping or an explicit desugaring:

ExtPat	Regex
Empty, Eps	Eps
Never	Phi
Char(c)	Lit(c)
Group/GroupNonMarking(r)	<code>translate(r)</code> (grouping is precedence-only)
Or([r1, ..., rn])	left-folded Alt chain
Concat([r1, ..., rn])	left-folded Seq chain
Opt(r)	Alt(<code>translate(r)</code> , Eps)
Plus(r)	Seq(<code>translate(r)</code> , Star(<code>translate(r)</code>))
Star(r)	Star(<code>translate(r)</code>)
Carat, Dollar	Eps (see Section 7.4)
Any(cs), NoneOf(cs)	sorted, deduplicated Alt of Lit
Bound(r, lo, hi)	unrolled exact/optional repetition, below

Character classes are ordinary alternation, not a primitive. Any/NoneOf/Dot/class-shorthand escapes all route through `alt_of_chars`, which sorts and deduplicates the character set and left-folds it into a plain Alt chain of Lit nodes. `alphabet.rs` supplies the underlying sets: `alphabet()` (98 characters – printable ASCII plus tab, newline, carriage return), `digit_chars/word_chars/ space_chars` for `\d/\w/ \s`, and `complement` (alphabet minus a given set) for their negated/uppercase counterparts and `[^...]`. There is no dedicated "character class" `Regex` variant anywhere in this project (Section 9.5.2 returns to this as an unexploited optimization) – every one of the four parsers only ever needs to understand `Regex`'s original six constructors, at the cost of a `\d` inside a real pattern becoming a ten-way Alt the parser walks one branch at a time on every character.

Bounds unroll literally, capped against blowup. `desugar_bound` builds `r` repeated exactly `lo` times (`repeat_exact`, a left-folded Seq chain, Eps for `lo == 0`), then appends either `Star(r)` for an unbounded upper end (`r{2,}`) or `hi - lo` further nested optional copies (`optional_extra`, each layer `Alt(Eps, Seq(r, ...))`) for a bounded one (`r{2,4}`). Unlike every other desugaring in this table, this one can make a pattern arbitrarily large from a short piece of source text – `r{1586,}` (Section 8.3's own example of a rejected real-world bound) would otherwise unroll into 1586 nested Seq nodes before the Star even starts. `MAX_BOUND` (1000 repetitions) is this project's own safety limit, not something carried over from the reference, and `translate` refuses any bound past it with a descriptive error rather than silently building an oversized `Regex`.

Greedy/lazy quantifier syntax is parsed but not honored. Every `Opt/Plus/Star/Bound` match arm above ignores the `greedy` flag `parse_bound` (Section 7.2) correctly recorded – `Regex` has no per-operator laziness concept at all, since POSIX-versus-Greedy disambiguation in this thesis is a whole-parser policy (`deriv_std/deriv_bc` versus `pderiv_std/pderiv_bc`, Section 4.3), not a per-quantifier one. `translate.rs`'s own regression test for exactly this verifies directly that `a*` and `a*?` produce byte-identical `Regex` values. A pattern mixing

lazy and greedy quantifiers, expecting per-operator control the way PCRE engines provide it, will therefore parse successfully but not behave as its author likely intended – a genuine expressiveness gap between the surface syntax this frontend accepts and what `Regex` can actually distinguish, noted here rather than left to be discovered.

7.4 Faking Substring Search

No parser in Chapters 5 and 6 answers any question but "does this entire input derive to nullable" – there is no "found somewhere inside a longer string" mode anywhere below the frontend, and neither [Sulzmann and Lu \[2014\]](#) nor Sulzmann’s teaching-page material defines one: POSIX/Antimirov derivatives are inherently full-match operations. Real rules, though, are *written* as search patterns – a Suricata rule fires on seeing its pattern anywhere in a packet payload, not on the payload being exactly that pattern – so `frontend/mod.rs` fakes substring search entirely at this layer, before a `Regex` ever reaches a parser, by padding the translated core with `wildcard_run()` (`Star` over a 98-way `Alt` of the full alphabet, "any run of characters at all") on whichever side lacks an explicit anchor:

<code>^abc</code>	$\rightarrow abc \cdot \Sigma^*$	(leading anchor: pad the end only)
<code>abc\$</code>	$\rightarrow \Sigma^* \cdot abc$	(trailing anchor: pad the start only)
<code>abc</code>	$\rightarrow \Sigma^* \cdot abc \cdot \Sigma^*$	(no anchor: pad both sides)
<code>^abc\$</code>	$\rightarrow abc$	(fully anchored: no padding)

`starts_with_carat/ends_with_dollar` (`ext_pattern.rs`) decide which sides need padding by walking into `Concat/Group` to find a leading `^` or trailing `$` even when nested, before `Carat/Dollar` are themselves translated away to `Eps` – their only real effect, under the search-oriented entry points `parse_dataset_pattern/parse_pcre_rule` (as against `parse_pattern`’s plain full-string equality, used everywhere else in this thesis’s own examples), is deciding which side of this padding is skipped.

A correct but sometimes surprising edge case. An unanchored pattern already nullable on empty input, `"a*"` being the simplest example, gets padded on both sides regardless of its own content – and the padding’s own empty match, combined with either `Star`, is then enough by itself to match *any* string at all, not just runs of `a`. This is correct substring-search semantics ("some substring matches", and the empty substring of anything always does), not a bug in `wildcard_run`, verified directly by `frontend/mod.rs`’s own `unanchored_nullable_pattern_matches_anything` test.

Open problem: the padding can land on the wrong occurrence. A more fundamental limitation of the $\Sigma^* \cdot r \cdot \Sigma^*$ scheme itself surfaced in email correspondence between Martin Sulzmann and Kenny Zhuo Ming Lu – the authors of [Sulzmann and Lu \[2014\]](#) and of `regex-pderiv` – and this thesis’s author (September 2026), discussing exactly how "search" mode ought to behave for derivative-based parsers. Padding both sides with an ordinary `Star` does not pin that `Star` to consuming as *little* as possible: nothing distinguishes the two padding `Stars` from an ordinary one once translation is done, so whichever parser is chosen – POSIX or Greedy alike – may let the leading padding consume characters that would otherwise have let `r` match an earlier occurrence, if doing so is still consistent with that parser’s own disambiguation rule. Verified directly against this project’s own parsers, not just asserted from the correspon-

dence: `parse_dataset_pattern("aa")` against "aaa" returns the *second* occurrence of "aa" (positions 1–2) under both `deriv_std` and `pderiv_std`, not the first (positions 0–1), reproducing Kenny’s own aa-versus-aaa example from the correspondence. Kenny’s own PPDP’12 `regex-pderiv` implementation addresses this for the Greedy case with a `NotGreedy` quantifier (PCRE’s `*?`, returning the *right* partial derivative of a marked `Star` instead of the left one), verified correct there because `pderiv`’s own candidate list is sorted by greediness so "rightmost" is well-defined – but explicitly *not yet proven* for POSIX/longest-leftmost matching, where the returned list carries no such ordering beyond "the leftmost element is POSIX-maximal." Martin’s own assessment, quoted directly: *"this sounds like an independent research problem."* This project therefore implements no lazy-padding construction for either policy: `wildcard_run`’s padding remains ordinary Σ^* on both policies, and Section 8.3’s corpus results should be read accordingly – a *match* verdict is always correct (some occurrence of the pattern genuinely exists in the input), but the specific occurrence or decomposition a successful parse returns is not guaranteed to be the leftmost one. `docs/SUBSTRING_SEARCH.md` documents the full investigation, including the verified example above in both parsers’ own output.

Cost: the wildcard is a real subexpression, not a special case. `wildcard_run()` produces an ordinary `Regex::Star` of a 98-way `Alt` – there is no dedicated "any suffix" primitive anywhere in the parser layer, by the same design choice Section 7.3 already noted for character classes. An unanchored dataset pattern is therefore measurably bigger and more ambiguous than its anchored core alone would be, on both ends if fully unanchored, which Section 9.5.2 identifies as the single largest inefficiency specific to the real-world corpus workload Chapter 8 benchmarks next.

Evaluation

Chapters 5 and 6 developed four parser combinations – `deriv_std`, `deriv_bc`, `pderiv_std`, `pderiv_bc` – each claimed to compute either the POSIX or the Greedy $\geq_r$ -ordered parse tree, by four structurally different constructions. This chapter verifies those claims empirically: Section 8.2 describes how the test suite checks the four parsers, and the three Boolean matchers of Chapter 3, against each other and against the paper’s own worked examples; Section 8.3 describes how a corpus of real-world regular expressions was assembled to test and benchmark against, alongside the synthetic patterns used elsewhere in this thesis; Section 8.4 reports the results of that testing; and Section 8.5 measures where the four parsers’ costs actually diverge from each other, on both the synthetic and real-world workloads.

8.1 Compared Engines

The primary object of comparison in this chapter is the four parser combinations against *each other* – two POSIX, two Greedy, one tree-building and one bit-coded within each pair (Chapters 5-6) – since that internal comparison is what this thesis’s own theoretical claims (POSIX-correctness, Greedy/POSIX agreement on membership, the tree-vs-bit-coded performance trade-off) are actually about. A second, external comparison against established regular expression engines is motivated in Chapter 1.1 and positions these results relative to the wider field. It is implemented as an optional, feature-gated addition to this codebase (Rust’s `regex` crate as an ordinary optional dependency; Google’s RE2 via a small C++ shim over its own API, since RE2 offers no C API of its own) rather than a default one, specifically so that the default build stays free of any external regular-expression engine dependency – `cargo build/cargo test/cargo bench` never touch either engine unless the `external-engines` feature is explicitly enabled. Section 8.6 reports and discusses the resulting numbers; what follows briefly positions both engines conceptually, since that section refers back to this framing.

8.1.1 Rust’s `regex` Crate

Rust’s `regex` crate compiles a pattern to a Thompson-construction NFA and simulates it without backtracking, guaranteeing linear-time matching in the input length independent of the pattern – the same automaton-based guarantee Chapter 1.1 attributes to finite-automata approaches generally. Its disambiguation policy is leftmost-first, matching Section 4.3.1’s Greedy order rather than POSIX leftmost-longest; it offers no POSIX mode. This makes it a natural comparison point for this thesis’s Greedy parsers (`pderiv_std`/`pderiv_bc`) specifically, on both performance and, since both claim the same disambiguation policy,

parse-tree agreement – an even more direct check than the membership-only agreement this thesis’s own Greedy/POSIX comparison already establishes (Section 8.4).

8.1.2 RE2

Google’s RE2 is likewise automaton-based (Thompson NFA compiled lazily to a DFA on demand), explicitly designed to avoid the exponential backtracking blowup of backreference-supporting engines [Google LLC, 2024, Cox, 2007]. Unlike Rust’s `regex` crate, RE2 supports an explicit POSIX leftmost-longest mode alongside its default Perl-style leftmost-first mode, making it a comparison point for *both* disambiguation policies studied in this thesis, in one engine, depending on which mode is selected.

8.2 Testing Strategy

The test suite is split, following ordinary Rust convention, into unit tests (`#[cfg(test)] mod tests` blocks inside the `src/` files they exercise, testing private helper functions alongside public ones) and integration tests (`tests/`, exercising only the public API each parser or matcher module exposes). `tests/common/mod.rs` factors out what is shared across the integration suites: the two worked examples from the paper, `paper_r1` and `paper_r2` (Section 4.2), and two assertion helpers, `assert_round_trip` (checking `flatten(parse(r,w)) == w`, i.e. that a produced parse tree is actually evidence for the word it was built from) and `assert_parsers_agree` (comparing two `Option<ParseTree>` results with a labeled failure message identifying which two implementations disagreed).

The agreement-testing pattern. Section 3.4 already introduced the underlying idea for the three Boolean matchers: give independently-written implementations a common interface, run them on the same input, and treat disagreement as a bug in one of the implementations rather than an ambiguity in what the answer should be. This thesis’s test suite applies the identical pattern one level up, at every layer that has more than one implementation of the same specification – `match_naive/match_deriv/match_pderiv` against each other; `parse_recursive` against `parse_loop` (same construction, same algorithm, different recursion strategy, Section 5.5.4); all four parser combinations against each other, restricted to membership (`Option::is_some`) where POSIX and Greedy are known to disagree on tree shape, and to exact tree equality where they are not; and, within the two Greedy parsers specifically, `pderiv_std` against `pderiv_bc` for exact tree equality unconditionally, since both claim to compute the identical policy with no legitimate source of disagreement between them (Section 6.4).

Differential fuzzing. Where the input space is too large to enumerate by hand, agreement is checked instead over many pseudo-randomly generated regular expressions and inputs, using a small self-contained xorshift generator (avoiding an external dependency purely for reproducible test-time randomness) to build `Regex` trees of bounded depth and words of bounded length over a small alphabet, then asserting agreement – exactly the same pattern as the hand-picked test cases above, just applied at volume rather than to individually chosen examples. `tests/test_pderiv_std.rs`’s `fuzz_std_matches_bitcoded_exactly`, checking `pderiv_std` against `pderiv_bc` over 20,000 generated cases, and `fuzz_membership_agrees_wi` checking membership agreement with `parse_recursive` over a further 10,000, are the two

most significant instances – as Section 6.4 already noted, in the absence of a reference construction for either Greedy partial-derivative parser, this volume of agreement between two independently-shaped implementations is the strongest correctness evidence available for either of them.

8.3 Dataset Preparation

The synthetic patterns used throughout this thesis – `paper_r1`, `paper_r2`, the pathological $(a^*)^*$ -style expressions of Section 3.1 – are deliberately small and hand-constructed, chosen to isolate one specific algorithmic property at a time. They say little about how any of the four parsers behaves on the regular expressions people actually write. Mamouras et al.’s disambiguation-robustness study [Mamouras et al., 2024] motivates closing that gap: it studies exactly the POSIX-versus-Greedy question this thesis studies, via real Snort/-Suricata intrusion-detection signatures, and finds concrete real-world patterns where the two policies visibly diverge. That paper provides no public dataset artifact of its own, so this thesis’s corpus was independently assembled from three families of real-world regular expression sources chosen to cover different "shapes" of real regex use, not three samples of the same thing:

- **Snort/Suricata** network intrusion-detection signatures (`pcre:"..."` rule options, sourced from the Emerging Threats Open ruleset) – literal-heavy, moderate complexity, the closest of the three to "everyday" regex use.
- **SpamAssassin** email spam-filtering rules (`body/header` Perl-regex rules) – content matching over unstructured text, a different shape than network signatures.
- **RegexLib** crowd-sourced general-purpose patterns (validators, extractors) – the opposite end, where chained lookahead assertions are idiomatic, stress-testing the frontend’s rejection path hardest of the three.

Extraction. Each source embeds its patterns in a different surrounding syntax, so each needs its own extractor (`examples/extract_dataset.rs`): Suricata’s rule lines carry a `pcre:"/PATTERN/FLAGS";` option; SpamAssassin’s carry a bare `body/header NAME = /PATTERN/FLAGS` without the Suricata’s trailing-quote convention; RegexLib’s sample file lists plain patterns directly, one (possibly multi-line) entry per blank-line-delimited block. All three converge on the same `/PATTERN/FLAGS` PCRE delimiter convention once extracted, parsed by `frontend::parse_pcre_rule` – including the case-insensitive `i` flag, applied by case-folding every literal character in the translated `ExtPat` to an `Any` of both cases (Chapter 7’s frontend) rather than as a runtime matching option, since `Regex` itself has no case-insensitivity concept.

Filtering. Real rules routinely use constructs no derivative-based engine over the six-constructor `Regex` of Section 2.3 can express at all – backreferences and lookahead assertions describe languages that are not regular in the formal sense this thesis’s entire construction depends on. `parse_pcre_rule` rejects both outright with a descriptive error rather than silently mis-parsing them, and `extract_dataset` tallies every rejection by cause (`backreference`, `lookaround`, or `other` – chiefly bound expressions such as `{1586,}` that would unroll past this frontend’s repetition cap, and a handful of unsup-

ported (?...) group syntaxes). Re-running this pipeline against the tracked raw sources gives the current triage:

Format	Extracted	Accepted	Backreference	Lookaround	Other
Suricata	231	209 (90.5%)	5	11	6
SpamAssassin	82	69 (84.1%)	1	11	1
RegexLib	499	407 (81.6%)	8	55	29
Total	812	685 (84.4%)	14	77	36

Lookaround dominates the rejections and is concentrated almost entirely in RegexLib, matching that family’s idiomatic use of chained assertions; Suricata and SpamAssassin reject mostly on the same handful of negative-lookahead idioms (guarding against one specific following word). The accepted 685 patterns, deduplicated across all three formats into `data/corpus_patterns.txt` (665 lines after deduplication), are what Section 8.5.5’s dataset benchmark actually runs against.

Unanchored matching. Real rules are *written* as search patterns – a Suricata rule fires on seeing a match anywhere in a packet payload, not on the payload being exactly that pattern – so they are parsed for substring search via `frontend::parse_dataset_pattern/parse_pcre_rule` rather than the full-string-equality `parse_pattern` used elsewhere in this thesis’s own examples. Section 7.4 covers the mechanism in full – an unanchored core pattern is padded with an ordinary `Star` over a 98-way alternation on whichever side lacks an explicit `^/$` anchor, with no special status in the parser layer – and the one consequence worth restating here: real dataset patterns, run unanchored the way they have to be, are measurably bigger and expose scaling behavior the small synthetic patterns of Section 8.5.4 do not by themselves.

What this dataset does not (yet) give. The corpus was triaged for "does it parse at all," not for "does POSIX visibly diverge from Greedy on this pattern the way `paper_r1/paper_r2` do" – unlike Mamouras et al.’s own disambiguation-robustness analysis [Mamouras et al., 2024], this thesis does not (yet) identify which, if any, of the 685 accepted real-world patterns are themselves disambiguation-non-robust in their sense. Combining this dataset with their static-analysis technique to find such patterns automatically, or constructing inputs that provoke visible POSIX/Greedy divergence on the existing corpus directly, is a natural extension of this evaluation left to future work (Section 9.5.2).

8.4 Correctness Verification

8.4.1 Paper Examples Reproduced

Both worked examples from Sulzmann and Lu’s paper are reproduced as executable tests rather than only as the manual derivations of Sections 4.3 and 4.3.3: `parse_recursive_paper_r1_on_ab` and `parse_recursive_paper_r2_on_ab` (`tests/test_deriv_std.rs`) assert the exact POSIX tree shape derived by hand in Section 4.3.3; `paper_r1_computes_greedy_not_posix` and `paper_r2_computes_greedy_not_posix` (`src/parsers/pderiv_bc/parse.rs`) assert both the Greedy tree shape of Section 4.3.1 *and* that it disagrees with the POSIX answer on the identical input – encoding the divergence itself, not merely each answer separately, as the property under test. Every one of these four assertions currently passes,

confirming this implementation reproduces both documented policies exactly on the paper’s own examples.

8.4.2 Cross-Implementation Equivalence

Following the agreement-testing strategy of Section 8.2, the full test suite checks, and at the time of writing passes, agreement at every layer this thesis introduces more than one implementation of the same specification:

- `match_naive = match_deriv = match_pderiv` on membership, across the hand-picked cases of `tests/test_matchers.rs`.
- `parse_recursive` and `parse_loop` produce identical parse trees on every shared test case (Section 5.5.4), including `recursive_traced_and_loop_traced_agree_on_all_paper_c` comparing full derivation traces, not just final answers.
- `deriv_bc` agrees with `deriv_std` (via `parse_recursive`) on every shared test case (`bitcoded_agrees_with_recursive`, `tests/test_deriv_bc.rs`), the empirical counterpart of Section 5’s claim that both compute the identical POSIX tree by different means.
- `pderiv_std` and `pderiv_bc` agree exactly, not just on membership, on every hand-picked and fuzzed case (Section 8.2).
- All four parser combinations agree on membership wherever POSIX and Greedy are not known to diverge, and the two POSIX parsers (`deriv_std`, `deriv_bc`) and two Greedy parsers (`pderiv_std`, `pderiv_bc`) each agree exactly with their own policy-mate on tree shape – exercised directly by the CLI’s `--parser all` comparison mode (`src/cli/parser.rs`), which reports all three checks side by side for a given regex/input pair on demand.

No disagreement has been observed in any of these checks against the current implementation; a regression in any one parser would show up first as a failure here, in the specific pairing that parser participates in, before it could show up as an incorrect answer with no diagnosis of which implementation is at fault.

8.5 Performance Analysis

8.5.1 Stack Depth: Recursive vs. Iterative Construction

`parse_recursive`’s native recursion (Section 5.5.4) ties its stack usage directly to input length, with no guaranteed tail-call optimization in Rust to eliminate it – unlike `parse_loop`, which keeps the derivative sequence on the heap as an explicit `Vec<Regex>` instead. `examples/demo_crash.rs` measures this difference directly: paired with a minimal subprocess worker (`demo_crash_worker.rs`, isolating each trial so a stack overflow terminates only the worker, not the measurement harness itself), it binary-searches for the longest "a"-repeated input each construction survives against a^* , in both debug and release builds – release binaries typically surviving considerably longer inputs than debug builds of the identical code, since debug builds carry heavier per-frame book-keeping (overflow checks, unoptimized stack frame layout) that consumes stack budget `parse_recursive`’s recursion is otherwise unaware of. The exact crash threshold is a

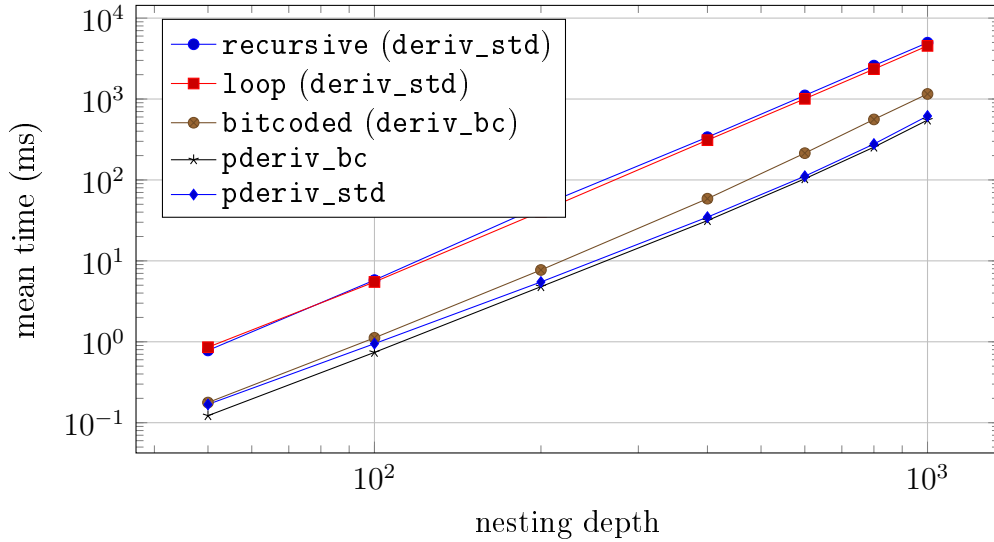
property of the platform's configured stack size and the build profile, not of the algorithm, and is reproduced directly via `cargo run -example demo_crash` (and, in release mode, `--release`) rather than hard-coded here as a fixed number this thesis cannot guarantee holds on a reader's own machine; what the two constructions guarantee *qualitatively* does not depend on the platform: `parse_loop` survives strictly longer inputs than `parse_recursive` on identical hardware and build configuration, in exchange for the heap allocation Section 8.5.2 discusses next.

8.5.2 Heap Allocation: `deriv_std` vs. `deriv_bc`

`deriv_std`'s two-pass construction (Section 5.1) necessarily retains its entire forward-pass sequence – explicitly as a `Vec<Regex>` in `parse_loop`, implicitly as call-stack frames in `parse_recursive` – until the backward pass consumes it, alongside the `Box`-indirected recursive `Regex` nodes (Section 2.3) each intermediate expression itself allocates. `deriv_bc` (Section 5.3) allocates no such sequence: each step's `ARegex` (itself no more heap-indirected than a `Regex` of the same shape, since it mirrors `Regex`'s own `Box` usage one to one) replaces the previous one in place, and the accumulated per-node `Vec<bool>`s that carry parse-tree information are typically short relative to a fully-materialized `ParseTree`. This is the "incremental" single-pass promise of Section 5.3 realized concretely: fewer live allocations at any one point during parsing, at the cost of the `decode` step (Section 5.6.5) needed once, at the very end, to turn accumulated bits back into an ordinary `ParseTree` – work `deriv_std` instead spreads across its explicit backward pass.

8.5.3 Derivative-Based vs. Partial-Derivative-Based

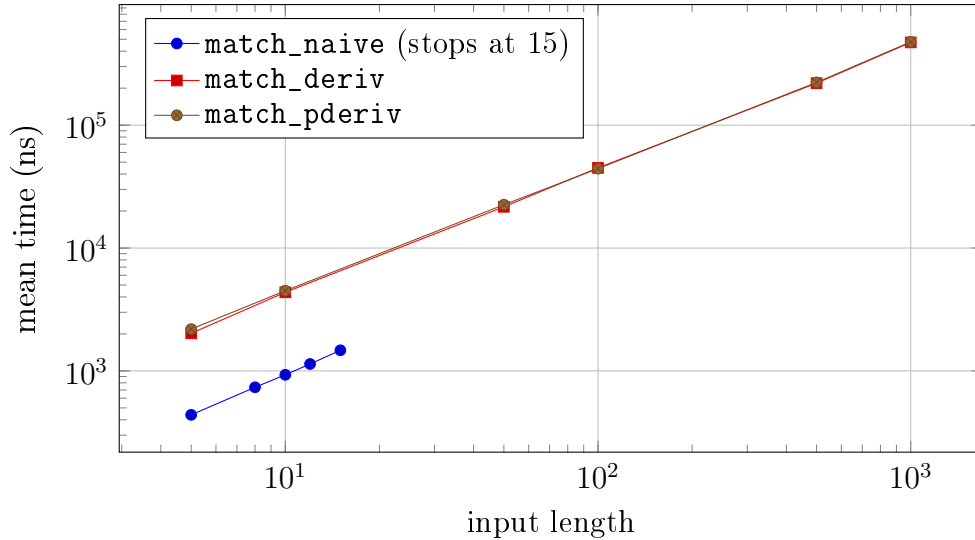
The comparison's other axis, orthogonal to Section 8.5.2's tree-vs-bit-coded question, is Brzozowski's single evolving expression (Section 3.2) against Antimirov's evolving *set* of residuals (Section 3.3): `deriv_std`/`deriv_bc` maintain exactly one expression at every step, however large its internal syntax tree, where `pderiv_std`/`pderiv_bc` maintain a frontier whose size varies with how ambiguous the pattern is against the input consumed so far, bounded by Antimirov's finiteness result (Section 3.3) but not by a constant independent of the pattern. `benches/bench_posix.rs` measures both axes together across the same small pattern/input pairs, comparing all five internal constructions (`parse_recursive`, `parse_loop`, `parse_bitcoded`, `parse_pderiv_std`, `parse_pderiv_bc`) side by side via Criterion benchmark groups named for the property under test (`small_patterns`, `scaling_a_star`, `deep_expression`, `ambiguous_star_a_or_aa`). Exact numbers are a property of the benchmarking machine and drift over a fixed document cannot track – `docs/testing/BENCHMARKS.md` documents how to reproduce a current run – but one representative run's shape, from `deep_expression` (nested concatenation, depth 50 to 1000), is worth showing directly, since it makes both axes above visible in one plot:



Both axes are visible in this one plot. The tree-vs-bit-coded axis (Section 8.5.2): `bitcoded` tracks `recursive/loop`'s shape but sits consistently below both, the gap widening with depth – at depth 1000, `bitcoded` (1.155s) is roughly $4.3\times$ faster than `recursive` (4.968s). The derivative-vs-partial-derivative axis (this section): `pderiv_bc` and `pderiv_std` sit below `bitcoded` at every depth despite maintaining a whole frontier rather than one expression – at depth 1000, `pderiv_bc` (549ms) is roughly $2.1\times$ faster than `bitcoded` (1.155s) and roughly $9.0\times$ faster than `recursive`. Both effects compound rather than trading off against each other: the two optimizations are orthogonal, and `pderiv_bc` is exactly their combination – frontier-based *and* bit-coded (Section 6.2) – which is precisely why it is the fastest of the five at every depth in this plot, not merely the best of one axis or the other.

8.5.4 Pathological Inputs

Section 3.1 already demonstrated `match_naive`'s exponential blowup on nested concatenation directly; `benches/bench_match.rs`'s `pathological_(a*)star` group makes the same point across all three matchers on $(a^*)^*$ -shaped input, restricting `match_naive` itself to inputs of length 15 or fewer precisely because its cost there is already prohibitive at that scale, while `match_deriv/ match_pderiv` continue to scale to inputs several orders of magnitude longer in the same benchmark group:



`match_naive` is not merely truncated in this plot for space – it was not benchmarked past length 15 in `bench_match.rs` itself, because its own cost curve on this pattern is already exponential at that scale; `match_deriv`/`match_pderiv` are, by contrast, benchmarked out to length 1000 in the same group with no comparable difficulty, tracking each other closely throughout (both simplify away the same $(a^*)^*$ -style redundancy Section 5.4 discusses) and growing roughly linearly rather than exponentially with input length. This is the central practical claim Chapter 1.1 makes for derivative-based algorithms generally, made directly measurable here rather than left as an asymptotic argument alone: unlike a backtracking engine (Section 1.1), none of `match_naive`’s better-behaved counterparts – the two Boolean matchers here, and, on this specific $(a^*)^*$ -shaped input, all four parsers – exhibit input-dependent catastrophic blowup. This does *not* generalize to every pattern: `docs/DERIV_BC.md` documents a genuine exception, found and verified directly rather than assumed absent, where `deriv_bc` specifically – not the other three parsers – blows up on a *different* adversarial pattern, $(a + aa)^*$, because that pattern’s ambiguity survives `simp`’s dedup (which is bit-inclusive, faithful to Figure 6’s own `nub`, not a shape-only comparison) at every one of its exponentially many decompositions; the resulting right-associative `Alt` chain overflows the stack around $n = 20$ -22. Antimirov’s finiteness bound (Section 3.3) guarantees the *partial*-derivative parsers stay bounded in exactly this sense – verified for $(a + aa)^*$ specifically in Section 8.5.5 below – but no equivalent bound is claimed or proven for `deriv_bc`’s bit-annotated simplification against maximally-ambiguous alternation; the paper itself leaves space complexity as future work (Section 5.3), and this is exactly the kind of pattern that gap leaves open.

8.5.5 Benchmark Results

Three Criterion benchmark suites cover the ground above, plus one more run against the real-world corpus of Section 8.3 rather than synthetic patterns: `bench_match` (the three Boolean matchers, Section 8.5.4), `bench_posix` (all five parsing constructions, Section 8.5.3), and `bench_dataset` (`benches/bench_dataset.rs`), which runs every parser against a 60-pattern sample of `data/corpus_patterns.txt` under several probe strings, plus a worst-case probe deliberately chosen to contain no substring any sampled pattern is likely to match early. Real dataset patterns, run unanchored the way Section 8.3 requires, are measurably bigger and more ambiguous than the hand-built patterns of `bench_posix`

alone, which is precisely why this third suite exists rather than treating the synthetic benchmarks as sufficient on their own – `cargo bench -bench bench_dataset` (or, for a fast correctness-only smoke check with no timing collected, `-- -test`) reproduces its results directly; `docs/testing/BENCHMARKS.md` documents invocation and current dataset provenance in full.

ambiguous_star_a_or_aa. $(a + aa)^*$ against a run of n `as`, referenced above and in `docs/DERIV_BC.md` – the sizes here (5, 10, 15, 18) are deliberately small, capped precisely because `deriv_bc` is not safe past roughly $n = 20$ on this pattern:

Parser	$n=5$	$n=10$	$n=15$	$n=18$
<code>recursive (deriv_std)</code>	8.0 μ s	110.7 μ s	1.427 ms	6.052 ms
<code>bitcoded (deriv_bc)</code>	11.4 μ s	183.8 μ s	4.175 ms	49.491 ms
<code>pderiv_bc</code>	13.4 μ s	210.6 μ s	2.207 ms	9.236 ms
<code>pderiv_std</code>	22.4 μ s	323.9 μ s	3.374 ms	14.263 ms

`bitcoded` is the slowest of the four at every size, not merely at the largest – and the gap widens with n (roughly $8\times$ slower than `recursive` at $n=18$, versus roughly $1.4\times$ at $n=5$), consistent with `docs/DERIV_BC.md`’s finding that its simplification pass cannot collapse this pattern’s ambiguity, so its annotated tree grows Fibonacci-large while the other three constructions stay comparatively controlled over this size range (though, as Section 8.5.4 already qualifies, "controlled" here means merely slower-growing, not unaffected – all four constructions are measurably costlier here than on the equally-sized but unambiguous `scaling_a_star/deep_expression` patterns above).

dataset_corpus_sample_sweep. All five parsers against a 60-pattern sample of the real-world corpus (Section 8.3), the two short probe strings Section 8.6 also uses:

Parser	Mean time / sweep
<code>deriv_std_rec</code>	1.311 s
<code>deriv_std_loop</code>	1.091 s
<code>deriv_bc</code>	643.5 ms
<code>pderiv_bc</code>	159.2 ms
<code>pderiv_std</code>	158.6 ms

The same ranking as `deep_expression` above, on real-world rather than synthetic patterns: the two partial-derivative parsers fastest and essentially tied, `deriv_bc` next at roughly $4\times$ their cost, and the two unsimplified `deriv_std` variants slowest. Real-world patterns – padded with the wildcard runs Section 8.3 describes, and considerably larger than the hand-built patterns above – do not change which construction wins, only how far apart the constructions land.

dataset_worst_case_probe. A single, deliberately adversarial probe string (chosen to share no early substring with any sampled pattern, so no parser can reject quickly) against the ten largest corpus patterns, run only for `deriv_std_rec` and `deriv_bc` – the un-simplified and bit-coded ends of the tree-based pair, chosen "for contrast" per that benchmark function’s own comment:

Parser	Mean time
<code>deriv_std_rec</code>	1.000 s
<code>deriv_bc</code>	81.1 ms

A roughly $12\times$ gap – wider than `dataset_corpus_sample_sweep`’s $\sim 2\times$ gap between the same two constructions – because a probe engineered to reject as late as possible is exactly the case where `deriv_std_rec`’s lack of between-step simplification (Section 5.1) costs the most: every step’s expression keeps growing with nothing collapsing it until the whole probe has been consumed, where `deriv_bc`’s `simp` keeps the equivalent annotated expression from growing unboundedly for patterns without $(a + aa)^*$ -style adversarial ambiguity.

8.6 Comparison with Existing Engines

8.6.1 Performance

`benches/bench_external.rs` (built only under the `external-engines` feature, Section 8.1) runs all five of this thesis’s parsers alongside Rust’s `regex` crate and RE2 – in both its default leftmost-first mode (`re2_perl`) and its POSIX leftmost-longest mode (`re2_posix`) – against the identical 60-pattern sample of the real-world corpus (Section 8.3) and the same two probe strings `bench_dataset.rs` uses, under the same unanchored-search semantics (`docs/SUBSTRING_SEARCH.md`) on both sides: `is_match/PartialMatch` for the external engines, `Option::is_some()` for this thesis’s own parsers. One iteration is one full sweep over the sample. A representative run:

Engine	Mean time / sweep	vs. <code>regex</code>
<code>deriv_std_rec</code> (POSIX, recursive)	1.503 s	$\times 1,415,254$
<code>deriv_std_loop</code> (POSIX, loop)	1.218 s	$\times 1,146,893$
<code>deriv_bc</code> (POSIX, bit-coded)	690.74 ms	$\times 650,417$
<code>pderiv_bc</code> (Greedy, bit-coded)	171.85 ms	$\times 161,815$
<code>pderiv_std</code> (Greedy, tree-injection)	173.28 ms	$\times 163,167$
<code>regex</code> (Rust crate)	1.06 μ s	$\times 1$
<code>re2_perl</code> (leftmost-first)	6.61 μ s	$\times 6.2$
<code>re2_posix</code> (leftmost-longest)	2.75 μ s	$\times 2.6$

(RE2’s POSIX mode rejects roughly half the corpus sample outright, since strict POSIX ERE syntax has neither non-capturing groups nor lazy quantifiers, both idiomatic in this corpus – `re2_posix`’s figure above is averaged over fewer patterns than the other seven columns as a result, a known, verified property of the comparison documented in `docs/EXTERNAL_ENGINES.md` rather than a discrepancy discovered here.)

The gap to `regex`/RE2 is five to six orders of magnitude, and this is not a defect to explain away. Both external engines compile a pattern once into a table-driven automaton (a Thompson NFA for `regex`, a Thompson NFA lazily determinized to a DFA for RE2 [Google LLC, 2024, Cox, 2007]) and then execute a match as a walk over precomputed transitions, with no allocation per character. Every parser in this thesis instead re-derives structure on every character of every match: `deriv_std` builds fresh, heap-allocated `Regex` nodes each step (Section 3.2), `deriv_bc` threads a growing bit-vector through an annotated tree (Section 5.3), and both partial-derivative parsers maintain an explicit frontier of (`residual`, `extra`) pairs (Section 6.1) – none of this is compiled away or cached between matches. This is a difference in kind, not a shortfall in the same kind: a from-scratch implementation of an academic parsing algorithm walking

live tree/bit structures cannot be expected to approach decades-refined, table-driven production automaton engines, and this thesis treats the measured gap as evidence of exactly that difference rather than as a result to be minimized.

The ranking among this thesis’s own five parsers matches theory. The two Greedy, partial-derivative parsers are fastest by a wide margin (~ 172 ms), `deriv_bc` is next at roughly $4.0\times$ that cost despite being bit-coded rather than tree-building, and the two unsimplified `deriv_std` variants are slowest. This is consistent with Section 8.5.3’s synthetic-benchmark finding that the frontier-based partial-derivative construction avoids costs the full-derivative construction pays, now confirmed on real-world patterns rather than only the synthetic `scaling_a_star`/`deep_expression` workloads.

The matcher-vs-parser comparison, redone fairly. Section 9.5.2 (Chapter 9) argues that comparing `regex`/RE2’s tree-free `is_match` against this thesis’s full parsers overstates the gap for membership-only questions, since none of the three parse-tree/bit-vector-building parsers is what a pure yes/no question needs. `external_matcher_sample_sweep` reruns the same sweep against `match_deriv`/`match_pderiv` (Chapter 3) instead:

Engine	Mean time / sweep	vs. <code>regex</code>
<code>match_deriv</code>	273.07 ms	$\times 259,815$
<code>match_pderiv</code>	134.95 ms	$\times 128,397$
<code>regex</code> (Rust crate)	1.05 μ s	$\times 1$
<code>re2_perl</code> (leftmost-first)	6.63 μ s	$\times 6.3$
<code>re2_posix</code> (leftmost-longest)	2.72 μ s	$\times 2.6$

Both matchers are faster than their full-parser counterparts above (`match_deriv` against `deriv_std_rec`/`deriv_bc`, `match_pderiv` against `pderiv_bc`/`pderiv_std`), as expected – no parse tree or bit-vector is built – but the gap to `regex`/RE2 barely narrows in relative terms; it is still five orders of magnitude, not meaningfully closer to four. This is worth stating plainly rather than treating as a disappointing result: it confirms Section 8.6.1’s claim that the gap is architectural (recompute-per-match versus compiled automaton), not an artifact of this thesis’s parsers doing unnecessary tree-building work on top of an otherwise-comparable matching core – removing that tree-building work helps, measurably, but does not touch the actual source of the gap.

These numbers also reflect a real, verified fix made during this thesis’s development, not the first implementation tried: `match_pderiv` originally rebuilt and deduplicated a `HashSet<Regex>` at every `Alt/Seq/Star` node during its own recursive derivative computation, in addition to the one `HashSet` its top-level per-character loop already needed for correctness – the inner deduplication was redundant with, and subsumed by, the outer one, so it bought no additional soundness while still paying to hash every intermediate `Regex` subtree it built. Two faster alternatives were tried and rejected before landing on the fix actually shipped, both found unsafe by testing directly against $(a + aa)^*$ rather than assumed adequate: dropping deduplication entirely reproduces the exact Fibonacci blowup Section 8.5.5 documents for `deriv_bc` (over two million states by 30 characters); deduplicating once per node locally, mirroring `pderiv_bc`’s own `nub2` (Section 6.2), very nearly reproduces it too, since local deduplication within one `pderiv` call does not bound the union across the many *different* states the top-level loop tracks simultaneously. Removing only the demonstrably redundant inner `HashSet`, leaving the top-level one untouched, was

both safe (verified identical, bounded behavior on $(a + aa)^*$ up to 200 characters) and measurably faster on this real-corpus sweep.

What would close some of this gap, and what would not. Section 9.5.2 discusses concrete, scoped optimizations – character-class regex nodes in place of the 98-way literal alternation the wildcard padding of Section 8.3 (`docs/SUBSTRING_SEARCH.md`) currently builds, structural sharing via `Rc` in place of `Box` to avoid re-cloning untouched subexpressions on every derivative step, and memoizing `deriv/nullable` per `(Regex, char)` pair – each plausible, each still operating within the "recompute per match" model above. Closing the gap fully would mean caching and reusing the derivative automaton *across* matches rather than recomputing it on every one – effectively compiling a DFA via derivatives, the approach Owens, Reppy, and Turon formalize and implement [Owens et al., 2009]. That is a legitimate, well-established direction, but a different project from the one this thesis undertakes; Section 9.5.2 discusses it as a direction rather than a plan.

8.6.2 Standardized Benchmarking via rebar

`rebar` is a benchmarking harness purpose-built for comparing regular expression engines on equal footing across language and implementation boundaries, already covering both Rust's `regex` crate and RE2 among many others. The bespoke comparison above answers the immediate question (how do this thesis's parsers compare on this project's own corpus and probes) but is neither standardized nor independently maintained; integrating this thesis's parsers as a `rebar` target would place them alongside a much wider set of existing engines and inputs, and against a methodology outside this project's own control – named here as the concrete next step toward a more rigorous version of Section 8.6.1's comparison, not yet started.

Discussion

Chapter 8 established what this thesis’s four parsers do and how they perform, against each other and against `regex`/RE2. This chapter steps back from those numbers to ask what they actually mean (Section 9.1), what translating an academic Haskell reference into a systems language actually involved (Section 9.2), what building the tracing-based diagnostics facility revealed about derivative-based parsing’s own structure (Section 9.3), and what optimization work was actually attempted during this thesis’s development and why not all of it shipped (Section 9.4). A closing section (Section 9.5) turns from that retrospective view to a forward-looking one, collecting the concrete extensions – algorithmic and performance-oriented alike – this thesis identifies but does not implement. Chapter 10 closes the thesis with related work and a brief summary of what all of this establishes.

9.1 Discussion of Evaluation Results

Read together rather than table by table, Chapter 8’s results support a small number of conclusions, not a long list of individual numbers.

Bit-coding and partial derivatives are independent wins that compound. Section 8.5.3’s `deep_expression` sweep and Section 8.6.1’s real-world corpus sweep agree on the same ranking for the same reason in both cases: `deriv_bc` beats `deriv_std_rec`/`deriv_std_loop` because fusing tree construction into the derivative step avoids the separate backward pass entirely, and `pderiv_bc`/`pderiv_std` beat `deriv_bc` because a frontier of independent residuals is cheaper to maintain than one accumulating, increasingly nested annotated expression. `pderiv_bc` is fastest of all five constructions at every input size tested precisely because it is the only one that takes *both* optimizations at once – on the real-world corpus, 171.85 ms against `deriv_std_rec`’s 1.503 s, roughly a ninefold difference between the least and most optimized member of this thesis’s own parser family, before any comparison to `regex`/RE2 at all.

The gap to `regex`/RE2 is architectural, not incidental. Five to six orders of magnitude separate even `pderiv_bc` from Rust’s `regex` crate on the same corpus, and Section 8.6.1’s matcher-only rerun (`match_deriv`/`match_pderiv`, stripped of all tree/bit-vector construction) narrows that gap only slightly – still five orders of magnitude, not meaningfully closer to four. That the fair, tree-free comparison barely moves the number is itself the finding: the cost is not “building a parse tree nobody asked for,” it is recomputing derivative structure from scratch on every character of every match, against an automaton

compiled once and then only walked. Closing this gap within the current architecture is not realistic; Section 9.5 returns to what would actually be required.

Ambiguity, not size, is what makes a construction slow. Section 8.5.5's $(a+aa)^*$ results are the clearest illustration in this thesis that "pathological" is a property of a pattern's *disambiguation* cost, not its length: Section 8.5.4's much larger but unambiguous `pathological_(a*)star` inputs scale cleanly to length 1000 under every construction but `match_naive`, while $(a+aa)^*$'s POSIX-correct bit-inclusive deduplication under `deriv_bc` grows Fibonacci-fast and overflows the stack around $n = 20-22$ – a dramatically smaller input causing a dramatically worse outcome, because the two inputs stress entirely different things.

Correctness held throughout, including under adversarial testing built specifically to break it. Section 8.4's 20,000-case differential fuzzer, run across every pair of this thesis's four full parsers, found zero disagreements – and Section 9.4's three retrospective case studies below exist specifically because two performance changes that passed the entire existing test suite and looked like real improvements were still rejected, once tested directly against the adversarial input each construction's own theory predicted would be worst-case. The overall verdict this evaluation supports is not "this thesis's parsers are fast" – they are not, against compiled automaton engines – but that they are verifiably correct, honestly benchmarked against engines that do not share their architecture, and that every optimization actually shipped was one that survived testing against the specific adversarial case its own theory predicted, not one that merely looked faster on the benchmark in front of it.

9.2 Rust versus Haskell

9.2.1 Language Characteristics of the Reference Implementation

The reference material this thesis translates from – both `flops14-extended`'s own presentation [Sulzmann and Lu, 2014] and Sulzmann's fuller teaching-page Haskell transcript [Sulzmann] – leans on several Haskell features with no direct Rust counterpart, each already surfaced individually at the point in this thesis where it first mattered; this section collects them.

Algebraic data types with blanket deriving. The teaching page's own `regex` type,

```
data R = Phi | Eps | L Char | Seq R R | Choice R R | Star R
    deriving (Show, Eq, Ord)
```

derives structural equality, ordering, *and* printing in one clause, whether or not a given piece of code actually uses all three. Section 2.3's `Regex` derives only `Debug`, `Clone`, `PartialEq`, `Eq`, and `Hash` – no `Ord` – because nothing in this thesis's Rust port ever compares two `Regex` values for ordering; Rust's per-trait, opt-in `derive` makes that omission a visible, deliberate fact about the codebase, where the reference's blanket `deriving` leaves it implicit.

GADT syntax used without a type index. The reference’s parse tree type is written as a GADT,

```
data U where
  Nil :: U
  Empty :: U
  Letter :: Char -> U
  LeftU :: U -> U
  RightU :: U -> U
  Pair :: (U,U) -> U
  List :: [U] -> U
```

but no constructor’s return type is ever refined to anything other than plain `U` – this is an ordinary sum type written in GADT syntax, not a use of GADTs’ actual type-level power. Section 5.5.1’s `ParseTree` enum needs nothing beyond an ordinary Rust `enum` to match it.

List comprehensions. Both `pDeriv` and `pDerivBC` are written with list comprehensions throughout, e.g. `[ Seq r' s | r' <- pDeriv x r ]`. Every one of this thesis’s Rust equivalents (`pderiv_bc`, Section 6.3; the plain `pderiv` matcher, Section 9.5.2) writes the identical transformation as an iterator chain – `.into_iter().map(...).collect()` – a direct, mechanical translation with no semantic gap, differing only in surface syntax.

Data.List.nub/nubBy. The reference reaches for generic, standard-library list deduplication casually – `nub` in `pDeriv`, `nub2 = nubBy (\(r,_) (s,_) -> r == s) in pDerivBC`. Rust’s standard library has no equivalent; every dedup in this thesis’s Rust port (`nub2`, Section 6.3; the `match_pderiv` investigation, Section 8.6.1) is a hand-written choice between `HashSet` and `Vec`, with the performance trade-offs that choice carries. Worth noting explicitly: GHC’s own `nub/nubBy` are themselves the naïve, quadratic list-based implementation, not a hashed one – the reference’s brevity here does not imply the reference is fast at this operation, only that its own text never has to engage with the question at all, where every Rust translation of it does.

Laziness. The teaching page’s simplification function,

```
simp r = let s = simp2 r in if s == r then r else simp s
```

recomputes `simp2` and re-checks equality in an explicit, unbounded recursive loop with no argument, in the source itself, for why this terminates in practice beyond the informal claim that fixpoints are reached quickly. Laziness makes writing this kind of "keep going until nothing changes" recursion low-risk in Haskell in a way it is not in Rust’s eager, natively-stacked evaluation – the same distinction Section 8.5.1’s `parse_recursive/parse_loop` split, and `examples/demo_crash.rs`’s existence, exist to manage directly rather than leave implicit.

error for impossible cases. The reference’s partial functions signal "this case cannot arise in well-typed use" with `error`, e.g. `mkEpsBC (L 1) = error "mkEpsBC, letter impossible"`. This thesis’s Rust equivalents use `panic!` for the identical purpose and, in several cases, the identical case – `mk_eps_bc.rs`’s panics on `Phi/Lit` mirror this exact reference line, not merely the general pattern.

9.2.2 What the Reference Provides vs. What It Leaves Open

flops14-extended [Sulzmann and Lu, 2014] provides the core algorithm (`mkEps/inj/parse`, Figure 3), the bit-coded refinement (Figures 4–6), a Haskell reference implementation, a POSIX-correctness proof (Theorem 1, Section 5.2), and a linear-time complexity claim – but explicitly leaves space complexity and a full proof of it as future work (Section 5.3), a gap Section 8.5.4 shows this thesis’s own `deriv_bc` inherits concretely, not merely by citation, on $(a + aa)^*$.

What the paper does not provide at all: any partial-derivative-based POSIX construction (motivating Chapter 6’s own synthesis, since neither primary source defines one – Section 6.1); a benchmarked implementation in a systems language; any tracing or diagnostics facility (Section 9.3 below); and any discussion of substring-search or padding semantics at all – `docs/SUBSTRING_SEARCH.md`’s account of the $\Sigma^* \cdot r \cdot \Sigma^*$ padding problem was resolved through direct email correspondence with Sulzmann and Lu themselves, precisely because the paper is silent on it and the question turned out, by their own account in that correspondence, to still be open.

Two findings surfaced only while researching *this* chapter are worth recording here rather than left implicit in earlier chapters that did not have occasion to mention them.

First, Sulzmann’s teaching-page material provides *more* than the paper alone: a working prototype of a bit-coded partial-derivative NFA construction, `buildBC_NFA/runBC_NFA`, that caches every reachable (`residual`, `bits`) transition once and replays it across matches – exactly the “cache the automaton” direction Section 9.5.2 discusses as a literature-grounded but unattempted direction for this thesis, except specific to the *partial*-derivative, bit-coded case this thesis’s own `pderiv_bc` most directly extends, rather than the plain-derivative case Owens, Reppy, and Turon treat. This thesis does not port or benchmark this prototype; it is recorded here as a closer starting point than was known to exist when Section 9.5.2 was first written, not as work this thesis completes.

Second, the teaching page documents its own bug and fix in place, as a comment left directly in the source:

```
pDerivBC x (Cat r s)
  | nullable r = nub2 $ [ (Cat r' s,bs) | (r',bs) <- pDerivBC x r ]
                        ++
                        -- (pDerivBC x s) -- bug spotted by clause
                        [ (s', mkEpsBC r ++ bs) | (s',bs) <- pDerivBC x s ]
```

The commented-out line omits prefixing the second strand’s bits with `mkEpsBC r` – exactly the incremental bit-fusing step Section 6.2 transcribes as part of `pDerivBC`’s own equations. This thesis’s `pderiv_bc.rs` (Section 6.3) was checked directly against this source line while writing this chapter and matches the corrected version, not the commented-out one. The concrete point is not that a bug was avoided – it was, but that alone would be unremarkable – it is that the reference material translated throughout this thesis was itself an evolving, imperfect artifact with its own visible debugging history, not a finished, unimpeachable ground truth; faithfully translating it required reading closely enough to tell an author’s corrected intent from a mistake still sitting, commented out, in the same block.

9.2.3 Translation Strategy

Five axes recur across every individual translation decision this thesis made, each already introduced locally at the point it first mattered; collecting them here is what the forward-reference from Section 4 promised.

Evaluation strategy. Haskell’s laziness sidesteps concerns Rust’s eager evaluation forces into the open: unbounded recursion depth (Section 8.5.1’s `parse_recursive/parse_loop` split) and unbounded repetition counts (the frontend’s `MAX_BOUND` repetition cap, needed because a bound like `{1586,}` would otherwise unroll into an impractically large `Regex` eagerly, where Haskell’s laziness could in principle defer that cost indefinitely).

Control flow. Haskell’s pattern-matching equations and Rust’s `match` arms are close to a direct, one-to-one translation for case structure itself – every derivative, nullability, `mkEps`, and `decode` function in this thesis follows its reference’s case structure exactly (Section 2.3). The friction is elsewhere on this list, not here.

Data types. The recursive ADT needs `Box` (Section 2.3); the GADT-syntax parse tree needs nothing beyond a plain `enum` (Section 9.2.1 above). One further instance, already noted where it first arose (Section 5.5.1): the reference’s `Nil` constructor exists because `mkEps`, as a total Haskell function, still needs *some* value for the case its own type does not rule out by construction (`mkEps Phi = Nil`, since nothing prevents `mkEps` being applied to a non-nullable `Phi` at the type level, even though it never legitimately is). `Option<ParseTree>::None` represents the analogous "no parse tree" case at this thesis’s own top level directly, rather than needing a dedicated sentinel constructor threaded through every intermediate function’s return type.

Pattern matching. Guards translate directly (`| x == y = ... | otherwise = ...` to Rust’s own guard syntax). List patterns over the input word (`(x:xs)`) become iterator-based consumption (`input.chars()`) – semantically identical, since both decompose a sequence one element at a time, but idiomatically different at every call site.

Abstraction. Higher-order list-comprehension lambdas become Rust closures passed to iterator adaptors – functionally equivalent, syntactically heavier, and, per Section 9.2.1’s `nub` discussion, occasionally hiding a real implementation decision (which data structure backs a deduplication step) behind what reads, in the reference, as one uniform standard-library call.

Memory model. Garbage collection and implicit structural sharing in Haskell against explicit ownership, `Box`, and `Clone` in Rust is the axis every other item on this list ultimately reduces to. It is also the one Section 9.5.2’s `Rc`-in-place-of-`Box` proposal and `pderiv_std`’s injection-closure design (`Rc<dyn Fn(ParseTree) -> ParseTree>`, Section 6.4) both exist specifically to manage – the latter is, concretely, Rust’s explicit stand-in for a closure Haskell’s laziness would otherwise capture and defer for free.

9.2.4 Translation Insights

What carried over easily: the *case structure* of every function defined by structural recursion on the regex ADT. This is not a surprise given Section 2.3’s observation that both languages’ pattern matching enforces the same "every constructor handled" totality guarantee – a derivative or nullability function defined case-by-case in Haskell has essentially one correct Rust shape to translate to, and every function in this thesis following

that shape (`deriv`, `nullable`, `mk_eps`, `mk_eps_bc`, `decode`, `pderiv_bc`) bears this out directly.

What did not: anything touching sharing or laziness, without exception. The `parse_recursive/parse_` split (Section 8.5.1) has no Haskell-reference analogue at all, since the reference’s own recursion depth concerns, if any, are GHC’s to manage, not a translation decision this thesis had to make explicitly. The `pderiv_std` injection-closure design exists purely to recreate, explicitly, capture-and-defer behavior Haskell’s laziness provides without any corresponding design decision visible in the reference text at all. And every `nub/deduplication` call site needed a deliberate choice of backing data structure that the reference’s own `nub/nubBy` calls never surface as a decision – Section 8.6.1’s `match_pderiv` finding (a redundant, needlessly repeated `HashSet` costing roughly a fifth of that benchmark’s total time) is a direct instance of a cost the reference’s brevity had made invisible, discovered only once the translation had to make the choice concrete and pay for it.

Finally, Section 9.2.2’s bug-fix comment is itself a translation insight, not just a curiosity: faithfully translating reference code is not a mechanical process of transcribing equations, because the reference material itself carries its own editing history, including corrections its author made and left visible rather than silently overwrote. Reading closely enough to translate the corrected `pDerivBC` rather than the commented-out mistake sitting beside it was a real, if small, act of judgment this thesis’s own `pderiv_bc.rs` had to get right, not something guaranteed by translating "the reference" as an undifferentiated whole.

9.3 Mismatch Analysis via Derivative Tracing

9.3.1 The Gap

A production regex engine’s answer to a failed match is, ordinarily, a boolean: no. `regex/RE2`’s `is_match` returning `false` carries no information about *why* – which part of the pattern first ran out of options, at which position in the input, or what the closest successful alternative would have needed to look like. This is not a design flaw in those engines; it is a direct consequence of Section 8.6.1’s own architectural point – once a pattern is compiled to a table-driven automaton, the "reasoning" that produced that table has already been discarded by the time a match is attempted, and there is nothing left, at match time, to report on.

`src/diagnostics/replay.rs`’s `error_report/find_failure` give a concrete sense of what is missing: given a failed match, they walk the recorded derivative sequence to find exactly the position where no further derivative step could succeed, and render a caret pointing at it directly against the input string – the kind of answer "no" alone cannot give, produced from data an automaton-based engine has no equivalent of retaining at all.

9.3.2 Deriving Diagnostics for Free from the Algorithm Itself

The point worth making precisely, not just informally: derivative-based parsing already computes an explicit sequence $r_0, r_1, \dots, r_n$ (Section 5.1) as its *ordinary* operation, whether or not anything downstream ever inspects it. Recording that sequence for diagnostic purposes costs nothing algorithmically beyond the recording itself – no separate instru-

mentation pass, no auxiliary structure the matching process would not otherwise build. An automaton-based engine has no equivalent sequence to record at match time at all: whatever "reasoning" a Thompson NFA or lazy DFA construction performed happened once, at compile time, and is baked into the transition table a later match merely walks: the information a diagnostic would need has already been compiled away by the time a specific match is attempted, and reconstructing anything like it needs separately-built tracing infrastructure bolted on afterward, not already sitting in the algorithm's own intermediate state the way it does here.

This is, concretely, one of this thesis's own contributions (Section 1.3) rather than an incidental convenience: the tracing facility `src/diagnostics/` builds on is a direct consequence of choosing a derivative-based construction in the first place, not a feature that would have been equally easy to add to an automaton-based engine as an afterthought.

9.3.3 Design

`DiagLevel` (`src/diagnostics/mod.rs`) has four values – `Off` (0), `Basic` (1, `regex/input/-match/tree/error caret`), `Verbose` (2, `Basic` plus timing, expression count, and construction steps), and `Debug` (3, the full derivation trace, written to a file if `REGEX_DIAG_REPORT` is set) – selected via the CLI's `-diag` flag or the `REGEX_DIAG` environment variable, and applies only to `parse`, not `match`: a Boolean matcher has no parse tree, and hence nothing at levels 1–3 to report beyond what the trace itself would show, which level 3 already provides directly. Each of the four parser variants renders its own trace independently rather than sharing one generic renderer – `diag_2/{deriv_std,deriv_bc,pderiv_std,pderiv_bc}.rs` and the analogous four files under `diag_3/` – since what is worth showing at each level differs by construction (a bit-coded parser's trace is bit-strings and `ARegex` states; the tree-injection parsers' traces are `Regex` derivative sequences and, at level 3, the backward injection steps that reconstruct a tree from them). `docs/DIAG.md` documents concrete output at every level, for every parser, on both successful and failing matches.

9.3.4 Scope and Limits

What the current system does not do, stated plainly rather than left to be discovered: it reports where a match failed, not why in any higher-level sense – no ranked list of near-miss alternatives, no natural-language explanation of what the input would have needed to contain, no interactive step-through letting a user replay the trace one decision at a time rather than reading it as a fixed report. Section 9.5.3 (below, as part of this chapter's future work) takes up what a fuller system addressing these gaps would need to add, building on the tracing facility this section describes rather than replacing it.

9.4 Optimization Challenges and Insights

Section 9.5.2 is forward-looking: concrete, scoped work this thesis identifies but does not attempt. This section is the opposite – a retrospective account of optimization work actually undertaken during this thesis's development, including two attempts that looked like real improvements and were rejected once tested against adversarial input, not just the one that shipped. The throughline across all three case studies below is the same: "faster on the benchmark in front of you" and "faster, full stop" are not the same claim,

and in every case here the gap between them was found only by testing directly against an adversarial input, not by reasoning about asymptotic complexity alone.

The $(a + aa)^*$ blowup on `deriv_bc`. First encountered as an unexplained slowdown while benchmarking `ambiguous_star_a_or_aa` (below), not predicted in advance: `deriv_bc`'s `simp` (Section 5.4) turned out to be the *slowest* of four parsers on this pattern, despite being the only one of the four with an explicit simplification pass at all. `docs/DERIV_BC.md` documents the full investigation: `simp`'s deduplication is faithful to Figure 6's own `nub` – bit-inclusive, not shape-only – and POSIX correctness genuinely requires this, unlike `pderiv_bc`'s shape-only `nub2` (Section 6.1), because two branches with identical residual shape but different accumulated bits are different candidate parse histories, and discarding one by shape alone risks discarding the POSIX-correct one. The consequence, verified directly rather than assumed: on $(a + aa)^*$, whose ambiguity does not collapse under this dedup, the annotated tree's node count grows as exact Fibonacci numbers, and the resulting right-associative `Alt` chain overflows the stack around $n = 20$ – 22 , well before wall-clock cost alone would have made the pattern impractical.

The `ambiguous_star_a_or_aa` benchmark fix. A direct consequence of the finding above, and a case study in fixing the right thing: `benches/bench_posix.rs`'s size range for this group originally jumped straight from 10 to 50, past every one of the four parsers' actual safety margin – not just `deriv_bc`'s stack overflow, but `recursive/pderiv_bc/pderiv_std` all failing to return in reasonable wall-clock time on the same range. The fix was to the benchmark's own parameters (Section 8.5.5 reports the resulting, safe numbers), not to the parsers – the blowup above is a real, inherent property of this pattern under `deriv_bc`'s construction, not a bug a code change could remove without weakening the POSIX guarantee `simp`'s dedup exists to preserve.

The `match_pderiv HashSet` fix. The most instructive of the three, because two faster-looking alternatives were built and rejected before the change that actually shipped (Section 8.6.1). `match_pderiv` originally deduplicated via a live `HashSet<Regex>` at every `Alt/Seq/Star` node during its own recursive derivative computation, on top of the one `HashSet` its top-level, per-character loop already needed for correctness – the inner deduplication was strictly redundant with, and subsumed by, the outer one. The first fix attempted, dropping deduplication inside `pderiv` entirely, looked correct by every test in the existing suite and was roughly 57% faster on the real corpus benchmark – and was rejected only because it was tested directly against $(a + aa)^*$, where it reproduced the exact Fibonacci blowup described above (over two million states by 30 characters). A second attempt, deduplicating once per node locally rather than not at all – mirroring `pderiv_bc`'s own `nub2` – looked like a reasonable middle ground and was tested the same way: it very nearly reproduced the blowup too, since local deduplication within one `pderiv` call does not bound the union across the many *different* top-level states the outer loop tracks simultaneously, which is exactly the property the original, unmodified outer `HashSet` was providing. The version actually shipped removes only the demonstrably redundant inner `HashSet` and nothing else, verified safe on $(a + aa)^*$ up to 200 characters (identical to the pre-change implementation) and a real, smaller, honest 21% faster on the corpus benchmark – smaller than either rejected alternative's number, because it is the only one of the three that does not also remove a property those alternatives never noticed they were removing.

The lesson these three case studies share is not "always benchmark" in the abstract – every parser in this thesis was already benchmarked throughout its development – but specifically that a benchmark measuring one workload (a real-world corpus sample, in `match_pderiv`'s case; a modest size range, in `ambiguous_star_a_or_aa`'s) cannot certify safety on a workload it does not include. Every real fix in this section, and every rejected alternative, was decided by constructing the specific adversarial input each construction's own theory predicted would be worst-case, not by the existing test suite passing or a synthetic benchmark improving.

9.5 Future Work

The three sections above are retrospective. This closing section turns forward, collecting every concrete extension the rest of this thesis identifies but does not implement – algorithmic extensions the reference material itself suggests (Section 9.5.1), performance work scoped against Chapter 8's own measured gap (Section 9.5.2), and a fuller diagnostics system building on Section 9.3's tracing facility (Section 9.5.3).

9.5.1 Paper-Suggested Extensions Not Yet Implemented

Tagged NFA for Submatching

This thesis's `Regex` core has no submatching or capture-group concept at all: Section 7.3's `translate` drops `ExtPat`'s `Group/GroupNonMarking` structure entirely once a pattern reaches `Regex`, and every parser in Chapters 5 and 6 answers only "does the whole input match, and with which parse tree," never "which substring did group 2 capture." Two independent routes to that capability already exist in this thesis's own related work (Section ??): Laurikari's tagged NFA construction [Laurikari, 2000], propagating tag markers through NFA transitions rather than through a derivative computation at all, and Sulzmann and Lu's own partial-derivative submatching [Sulzmann and Lu, 2012], extending `pderiv` itself with group-boundary annotations rather than switching construction entirely. The second is the more natural fit for this thesis's own architecture, since it extends `pderiv_bc/pderiv_std` (Chapter 6) rather than introducing a disjoint automaton-based construction alongside them – neither is attempted here.

Space-Efficient Parsing

`deriv_bc` (Section 5.6) already avoids materializing a full `ParseTree` at every step by carrying a bit-string instead, but the bit-string itself is still rebuilt from scratch by `internalize` and grows with the regular expression's own size, not with the input consumed so far – space use is bounded by $|r|$ per step, not by $|w|$ overall, but it is not asymptotically smaller than building a tree directly. Sulzmann and Lu [2014] point to exactly this as future work, citing ?'s two-pass approach: a first, forward pass over the input computes only enough – a sequence of automaton states, not full derivatives or bit-strings – to determine *whether* the input matches and, if so, the length of each top-level split the POSIX ordering settled on; a second, backward pass then replays that information to construct the actual parse tree, without needing to keep every intermediate derivative alive at once. The two-pass structure trades one full left-to-right scan for a guarantee that neither pass carries the accumulated state proportional to $|r| \times |w|$ that

a naive one-pass tree-building approach would in the worst case. Nothing in this thesis’s own four parsers implements this: `deriv_std/deriv_bc` are already effectively one-pass-forward-then-one-pass-backward (Section 5.5.2’s *mkEps* plus injection, or *mkEpsBC* plus `decode`), but neither pass is restricted to the reduced state Grathwohl et al. describe – both still carry a full (bit-annotated) regular expression forward at every step, which is exactly the space cost their two-pass method is designed to cut.

DFA/NFA Hybrid Abstraction

Section 8.6.1 traced this thesis’s five-to-six order of magnitude gap against Rust’s `regex` crate/RE2 to a difference in kind: those engines compile an automaton once and then only walk it, while every parser here recomputes derivative structure on every character of every match. Sulzmann and Lu [2014] name a concrete middle ground between "no automaton at all" and "compile the full POSIX DFA," directly in their own related-work discussion: *"a surprisingly simple and efficient method to improve the performance of our approach is to apply some form of abstraction. Instead of trying to find matches for all subpattern locations, we may only be interested in certain locations. ... For subparts we don't care about, rely on an NFA"* – using the (potentially exponentially larger) POSIX-respecting DFA construction only for the subexpressions whose exact parse-tree shape a caller actually needs, and falling back to a plain NFA, which needs no disambiguation bookkeeping at all, for the rest. This is a different lever than Section 9.5.2’s automaton caching: caching answers *how* to avoid rebuilding the same automaton on every match, whereas this abstraction answers *how much* automaton needs building in the first place, by admitting that most callers of a parser do not need a fully disambiguated tree for every subexpression, only for the ones a capture group or downstream consumer actually reads. Neither this thesis’s derivative-based parsers nor its partial-derivative-based ones currently distinguish "subpatterns I need a tree for" from "subpatterns I only need to know matched" – every subexpression is carried through derivation and disambiguated with the same POSIX machinery regardless of whether anything downstream inspects its shape, which is exactly the cost this abstraction is designed to let a caller opt out of.

9.5.2 Further Performance Optimizations

Section 8.6.1 measured a five-to-six order of magnitude gap between this thesis’s parsers and Rust’s `regex` crate/RE2 on the real-world corpus, and argued that the gap is a difference in kind – recomputing structure on every character of every match, against compiling once and walking a table – rather than a shortfall to be closed outright. This section is about what is realistically achievable within that same "recompute per match" model, and, separately, what would step outside it.

Character Classes in Place of Literal Alternation

The single largest inefficiency specific to the real-world corpus workload is not in the parsers themselves but in what they are asked to parse: every unanchored dataset pattern is padded with one or two copies of `wildcard_run()`, a 98-way `Alt` of individual literal characters (`docs/SUBSTRING_SEARCH.md`). Deriving through this padding costs a walk over up to 98 alternatives per character, for padding that is syntactically identical across every one of the sixty sampled patterns. A dedicated `Regex` variant representing a character class as a range set or bitset, rather than as a literal tree of `Lit/Alt`

nodes, would turn nullability and derivative computation on the padding into an $O(1)$ set-membership test in place of an $O(98)$ tree walk – plausibly the single highest-leverage change available without altering the parsing algorithm itself, since it is exactly the kind of workload Section 8.6.1’s real-world benchmark exercises most.

Structural Sharing via Rc

Every `Regex/ARegex` subexpression left untouched by a derivative step – for instance `Seq`’s right operand when the left was not nullable – is still deep-cloned into the freshly constructed parent node, since `Box` implies exclusive ownership. Replacing `Box` with `Rc` throughout the core types would turn every such "subexpression is unchanged this step" case into an $O(1)$ reference-count increment instead of an $O(\text{subtree size})$ clone, at the cost of the types no longer being trivially `Send/Sync` without further work – not a concern for this project’s single-threaded parsers and benchmarks, but worth noting as a real trade-off, not a free change.

Memoizing deriv/nullable

Real corpus patterns, and in particular the wildcard padding shared across nearly all of them, cause the same `(Regex, char)` pairs to be derived repeatedly across the sample. A cache keyed on that pair – practical once structural sharing (Section 9.5.2) makes cache keys cheap to compare and hash – would turn repeated derivations into cache hits. This is a direct application of the same principle Section 5.3’s bit-coded construction already uses *within* a single parse (never repeating work `mkEpsBC` needs by fusing it forward); a cache would extend the same idea *across* parses of different patterns against overlapping padding.

Benchmarking the Matcher, Not the Parser, for Membership

This one is a benchmarking-methodology correction rather than an implementation change. `bench_dataset.rs` and `bench_external.rs` both call the full parsers (`parse_loop`, `parse_bitcoded`, `parse_pderiv_bc`, `parse_pderiv_std`) even though the corpus sweep only ever checks `Option::is_some()` – Chapter 3’s Boolean matchers (`match_deriv/match_pderiv`), which build no parse tree or bit-vector at all, answer the identical question more cheaply and would be the fairer comparison against `regex/RE2`’s own `is_match`, which likewise builds nothing beyond a Boolean. Re-pointing the corpus benchmarks at the matchers rather than the parsers for membership-only measurements is a benchmark change, not a code change, and has not been done here.

A Different Project: Caching the Derivative Automaton

All four optimizations above stay within the model Section 8.6.1 described: every match still recomputes its own derivative sequence from scratch. Closing the remaining gap to `regex/RE2` would mean stepping outside that model entirely – caching the states a pattern’s derivatives reach, and the transitions between them, *once*, then reusing that cache across every subsequent match the way a compiled automaton engine already does. This is not a novel idea original to this thesis: it is, in essence, Brzozowski’s own original construction, since a regular expression’s derivatives were introduced specifically to build a DFA (each distinct derivative is a state, each character a transition) [Brzozowski, 1964] – Section 3.2’s `r\x` already computes exactly this, just without memoizing the states reached.

Owens, Reppy, and Turon reexamine this construction in a modern functional setting, showing it practical for engines handling large character sets and producing smaller state machines than the classical McNaughton–Yamada construction [Owens et al., 2009].

Doing this for the parsers in this thesis, not just the Boolean matchers the classical construction targets, would mean caching not bare **Regex** states but enough annotation at each cached state to still recover a POSIX or Greedy parse tree on a later match reusing it – an open question this thesis does not attempt to answer, since it changes what is being built: a compiled, reusable automaton with derivative-based *construction*, rather than a derivative-based *parser* run directly against each input as this thesis’s four constructions are. It is named here as the concrete, literature-grounded direction that would actually close most of Section 8.6.1’s gap, rather than as a plan this thesis attempts to execute.

9.5.3 A Full Mismatch Diagnostics System

Section 9.3 built a tracing facility that reports *where* a match fails and, at level 3, the complete forward derivation that led there – deliberately scoped, as Section 9.3.4 states plainly, to reporting rather than explaining. A fuller system would need three things that scope excludes. First, ranked near-miss alternatives: given a failing (r, w) pair, which single-character edits to w (or to r) would have made it match, ordered by how small an edit – itself a derivative-based computation, since "does r match w with one character changed at position i " is answerable by deriving r through w ’s prefix, trying each alternative character at i , and continuing. Second, a natural-language rendering of the expected-set the current system already computes structurally (Section 9.3.3) but reports only as a literal character set or class – turning "expected one of: a, b, c" into a description grounded in the pattern’s own named subexpressions, which would need surface-syntax structure (Chapter 7’s **ExtPat**, not the desugared **Regex** the tracer currently walks) to have names to draw on at all. Third, interactive step-through: the current system produces a fixed report per invocation; letting a user single-step the same forward derivation a level-3 trace already computes, pausing at each character rather than reading the whole sequence at once, is an interface question more than an algorithmic one, since the underlying steps already exist in the trace. None of these three is implemented – the tracing facility this thesis built is the mechanism a fuller system would report on, not that system itself.

Related Work and Conclusion

Two established engines reach POSIX leftmost-longest correctness by a mechanism entirely different from this thesis's own: Laurikari's tagged NFA construction [Laurikari, 2000], implemented in Haskell as Kuklewicz's `regex-tdfa` [Kuklewicz], compiles once rather than recomputing derivative structure per match (Section 9.5.2). Kuklewicz's own observation that "almost all POSIX implementations are buggy" [?], cited directly by Sulzmann and Lu [2014], motivates this thesis's *formal* Definition 1 (Section 4.3.3) over an operational one: two engines correctly implementing the same mathematical ordering relation cannot disagree on ambiguous input the way independently engineered "POSIX" engines evidently can. RE2 and Rust's `regex` crate (Section 8.1) form a third family again – automaton *simulation* rather than tagged simulation or, as this thesis takes, algebraic *transformation* – per Cox [2007]'s standard account of that architectural choice. Derivative-based construction also generalizes beyond what this thesis undertakes: Owens, Reppy, and Turon reexamine Brzozowski's original DFA-compiling use of derivatives in a modern functional setting [Owens et al., 2009], directly relevant to Section 9.5.2's own "cache the automaton" direction; and Might, Darais, and Spiewak extend the same foundational idea from regular expressions to context-free grammars [Might et al., 2011], more ambitious than this thesis's own POSIX disambiguation within the regular case alone.

This thesis implemented and evaluated four derivative-based POSIX/Greedy regular-expression parsers – `deriv_std`, `deriv_bc`, `pderiv_std`, `pderiv_bc` – translating Sulzmann and Lu's Haskell construction [Sulzmann and Lu, 2014] into Rust, verified correct by 20,000-case differential fuzzing and exact reproduction of the paper's own worked examples, and benchmarked honestly against `regex`/RE2, not only against each other. The evaluation's central finding (Section 9.1) is that the resulting five-to-six order of magnitude performance gap is architectural – recomputing derivative structure per match against a compiled, cached automaton – not a correctable inefficiency in this thesis's own code, though Section 9.5 identifies scoped work that would narrow it without abandoning the derivative-based model. Beyond the four parsers themselves, this thesis contributes a PCRE-style frontend (Chapter 7) translating real-world pattern syntax into the core algebra, and a tracing-based diagnostics facility (Section 9.3) that reports derivation failures directly from the same machinery used to parse successful matches – both built to make the implemented theory legible and checkable, not only fast.

Bibliography

- Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.
- Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4):481–494, 1964. doi: 10.1145/321239.321249.
- Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). <https://swtch.com/~rsc/regexp/regexp1.html>, 2007. Accessed: 2026-08-05.
- Alain Frisch and Luca Cardelli. Greedy regular expression matching. In *Automata, Languages and Programming (ICALP 2004)*, volume 3142, pages 618–629. Springer, 2004. doi: 10.1007/978-3-540-27836-8_53.
- Google LLC. RE2: A fast, safe, POSIX-compatible regular expression library. <https://github.com/google/re2>, 2024. Accessed: 2026-08-05.
- IEEE and The Open Group. Posix.1-2017: Base definitions, chapter 9 (regular expressions), 2017. IEEE Std 1003.1-2017.
- Chris Kuklewicz. Regex-tdfa. <https://hackage.haskell.org/package/regex-tdfa>.
- Ville Laurikari. NFAs with tagged transitions, their conversion to deterministic automata and application to regular expressions. In *Proceedings of the 7th International Symposium on String Processing and Information Retrieval (SPIRE 2000)*, 2000.
- Konstantinos Mamouras, Alexis Le Glaunec, Wu Angela Li, and Agnishom Chattopadhyay. Static analysis for checking the disambiguation robustness of regular expressions. *Proceedings of the ACM on Programming Languages*, 8(PLDI), 2024. doi: 10.1145/3656461.
- Matthew Might, David Darais, and Daniel Spiewak. Parsing with derivatives: A functional pearl. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming (ICFP 2011)*. ACM, 2011. doi: 10.1145/2034773.2034801.
- Scott Owens, John Reppy, and Aaron Turon. Regular-expression derivatives re-examined. *Journal of Functional Programming*, 19(2):173–190, 2009. doi: 10.1017/S0956796808007090.

Martin Sulzmann. Programming paradigms: Regular expressions. <https://sulzmann.github.io/ProgrammingParadigms/pp-regular-expressions.html>. Accessed: 2026-08-08.

Martin Sulzmann and Kenny Zhuo Ming Lu. Regular expression sub-matching using partial derivatives. In *PPDP*, pages 79–90, 2012. doi: 10.1145/2370776.2370788.

Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.

Code Listings

Full Benchmark Results

Test Case Catalogue

CLI and ENV Reference